



香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

ELEC 3310

Digital Fundamentals and System Design

Wenchao Miao

Outline Lecture 10 Finite State Machines Counters



THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY

1 Basic Concepts

2 Counter Design

3 Synchronous Counter

4 Logic Minimization

5 Gray Code Counter

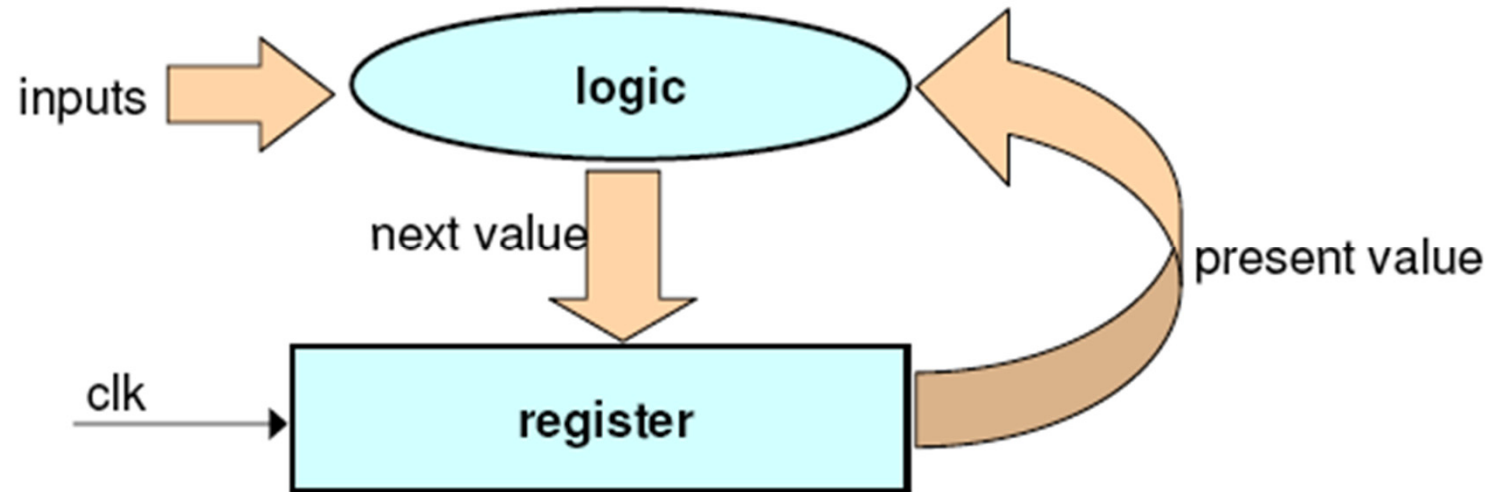
6 Special Counter

Main Content

1. Learning Outcomes

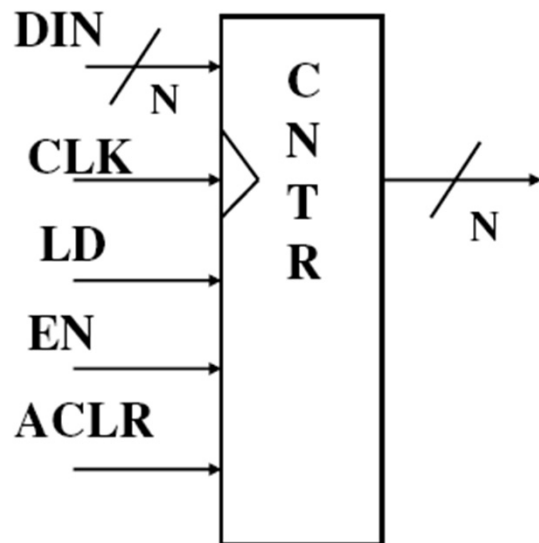
2. Basic Concepts of Counter

1. Design a counter that will have any specified sequence of states
2. Design binary UP/DOWN synchronous counters using **Toggle** mode
3. Design a BCD synchronous counter
4. Use Karnaugh map to minimize Boolean expressions **(SOP)**
5. Use Karnaugh map to minimize Boolean expressions **(POS)**
6. Design a 3-bit Gray Code synchronous counter
7. Design a special counter **with unused states** based on JK flip flops
8. Design a special counter **with unused states** based on T flip flops



- **Control inputs:**
 - Enable, up/down, reset, parallel load
- **A variety of count sequences:**
 - Binary, decimal, BCD, gray –code, pseudo-random

- **Very useful sequential building block.**
- **Example of usage**
 - **Generation of memory addresses**
 - **To count events**
 - E.g. count the number of times an event happens
 - Count time
- **Example features: synchronous load, count enable, asynchronous clear**

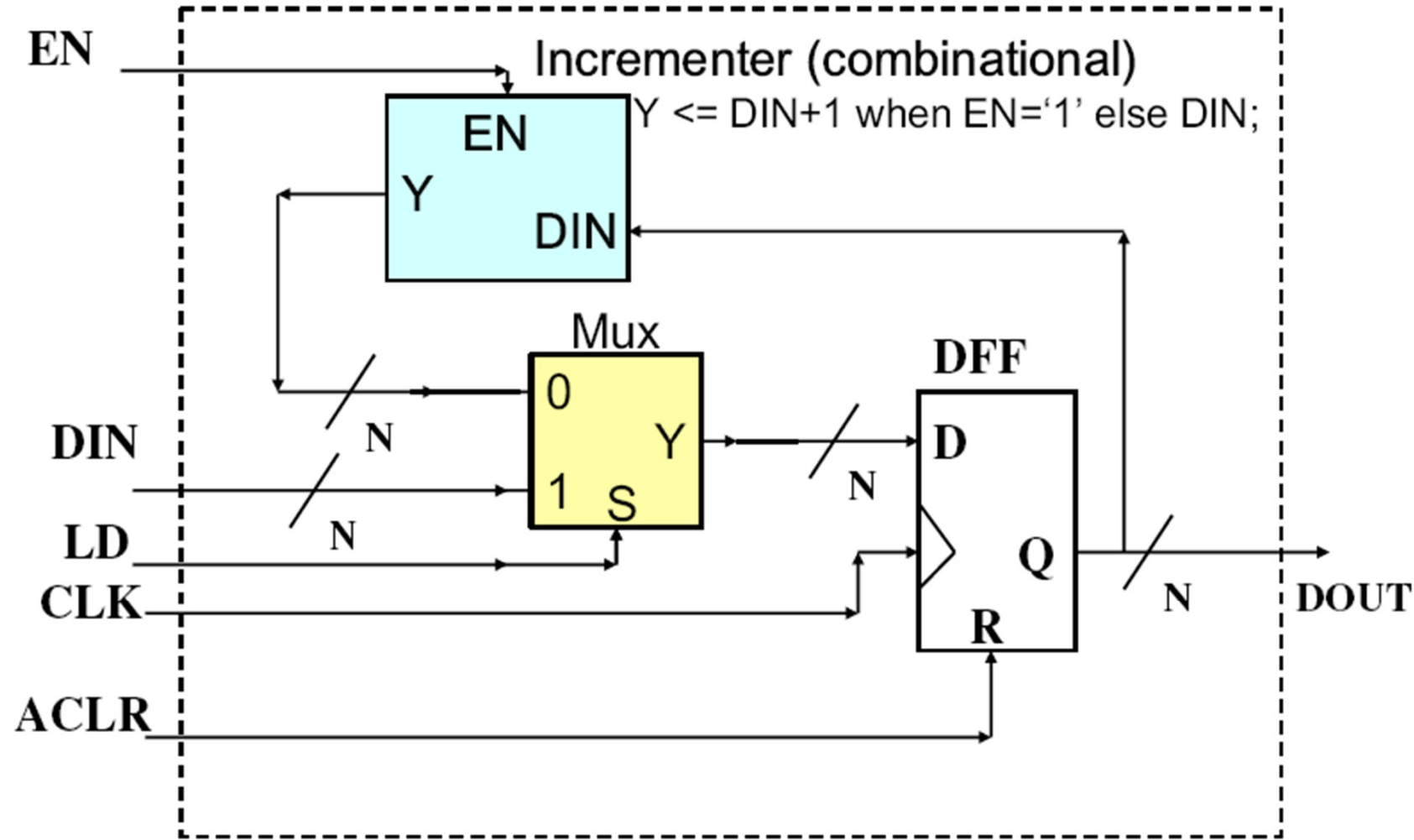


- ***LD*** asserted loads counter with ***DIN*** value on next active clock edge

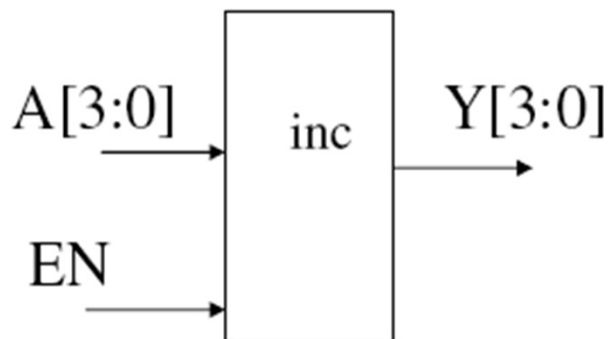
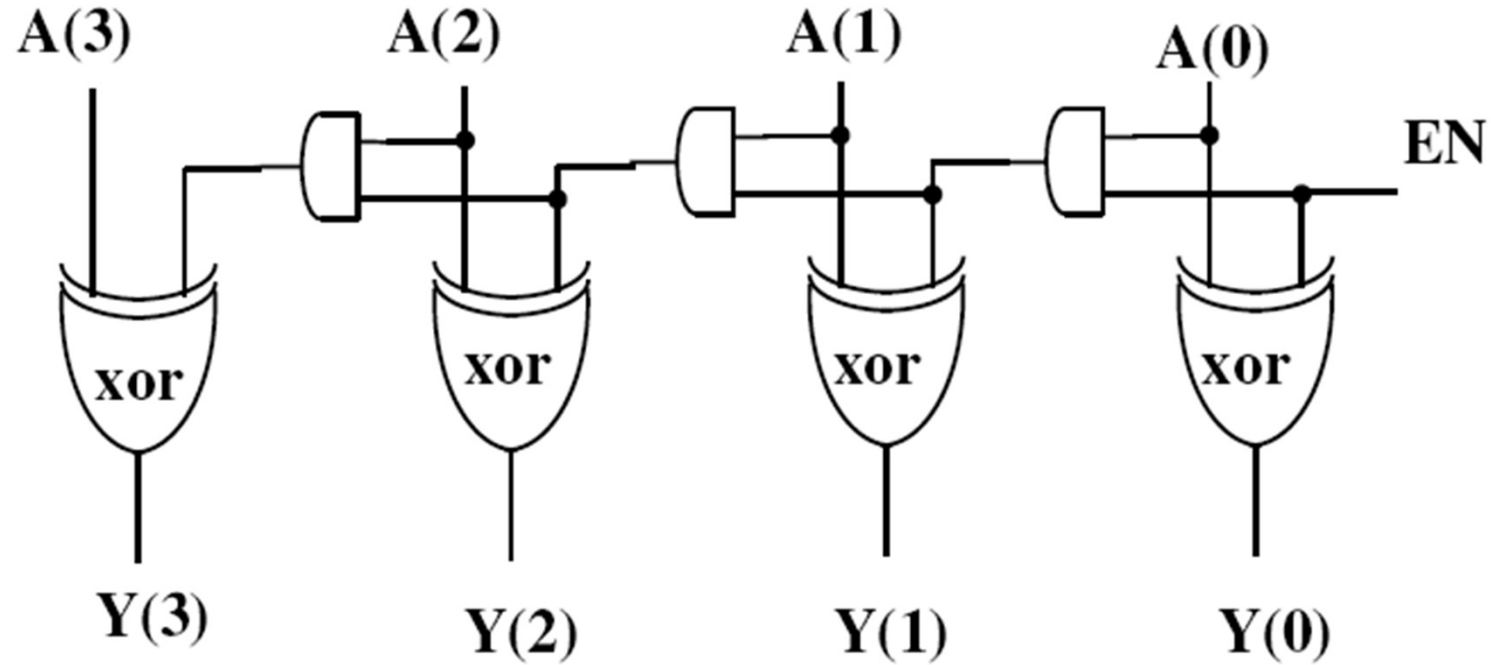
- ***EN*** asserted will increment counter on next active clock edge

- ***ACLR*** will asynchronously clear the counter

One way to build a counter – use incrementer and register



How to build an incrementor

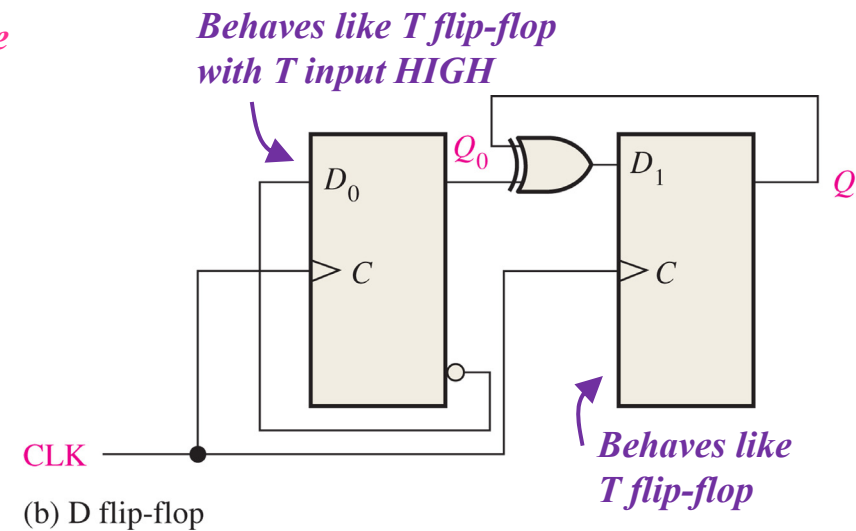
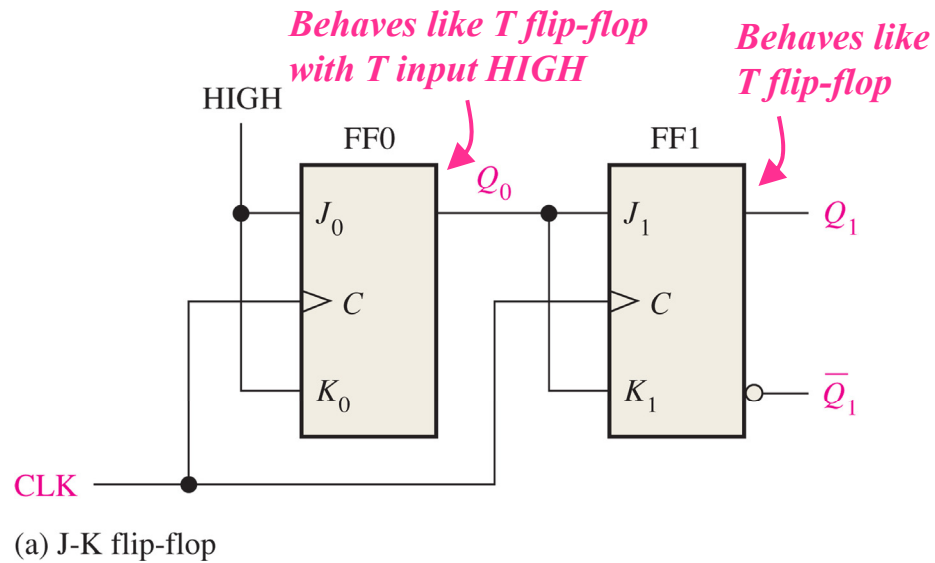


if $EN = 1$ then $Y \leq A + 1$
if $EN = 0$ then $Y \leq A$

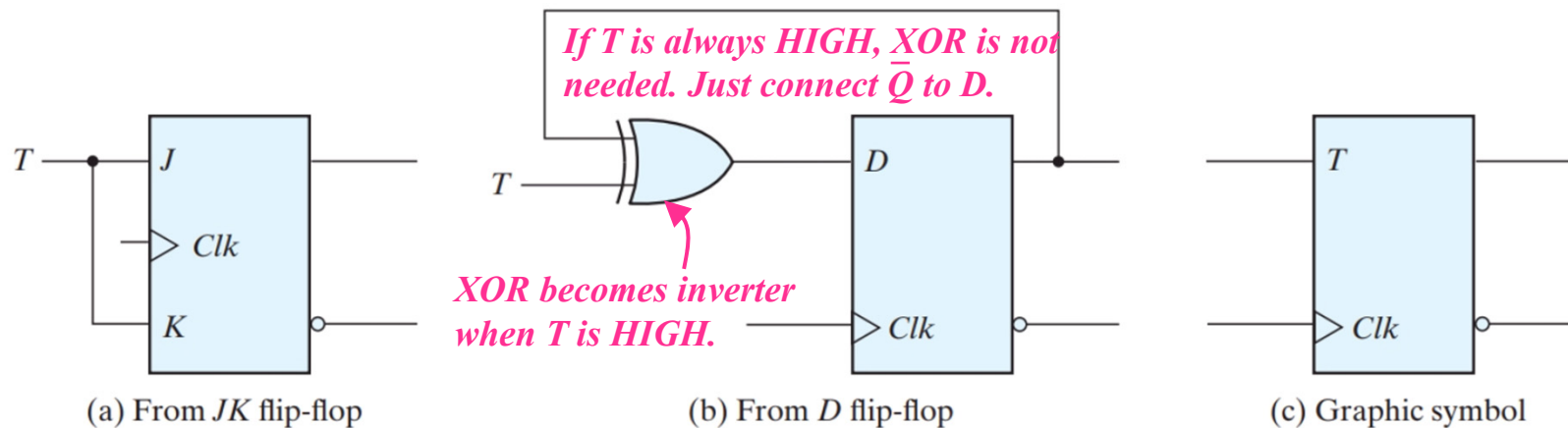
- 1. Obtain specification of the desired counter**
- 2. Create a state diagram from specification.**
- 3. Create a state table from state diagram.**
- 4. Derive the next-state logic and output logic (if any) expressions.**
- 5. Implement circuit described by expressions.**

- 1. You can also design a counter using Toggle mode**
 - **Specifically using T flip-flops**
 - **But you can also use JK or D flip-flops and configure them to work as T flip-flops**
- 2. Determine under what condition each bit of count value Toggles**
- 3. Connect a T flip-flop to satisfy that condition for that bit**

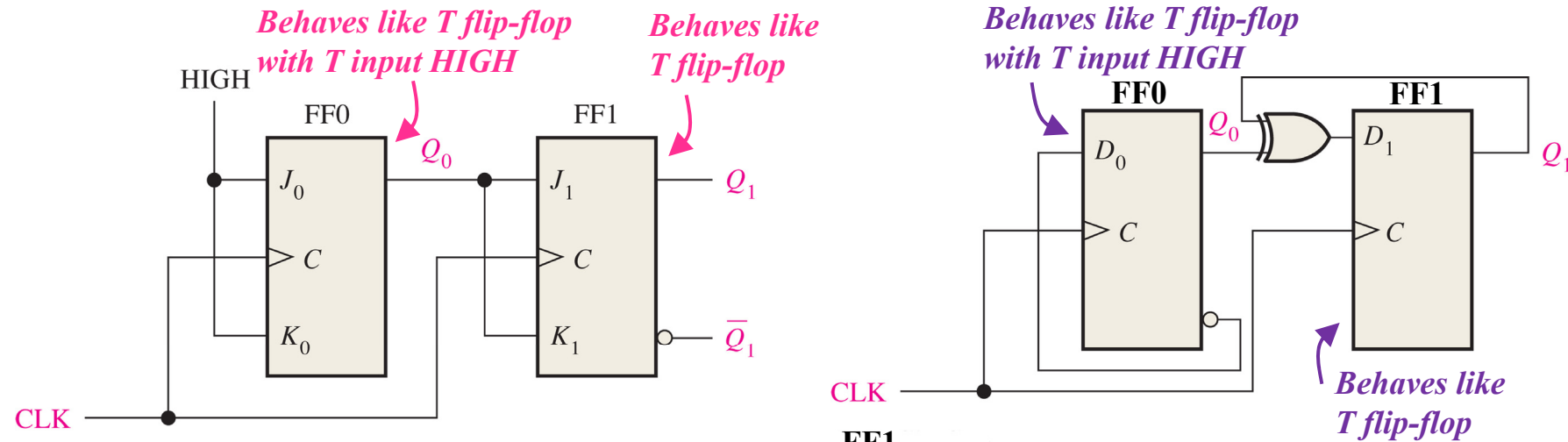
2-bit binary synchronous counters



Remember, JK flip-flop and D flip-flop connected as below behave like a T flip-flop



2-bit binary synchronous counters

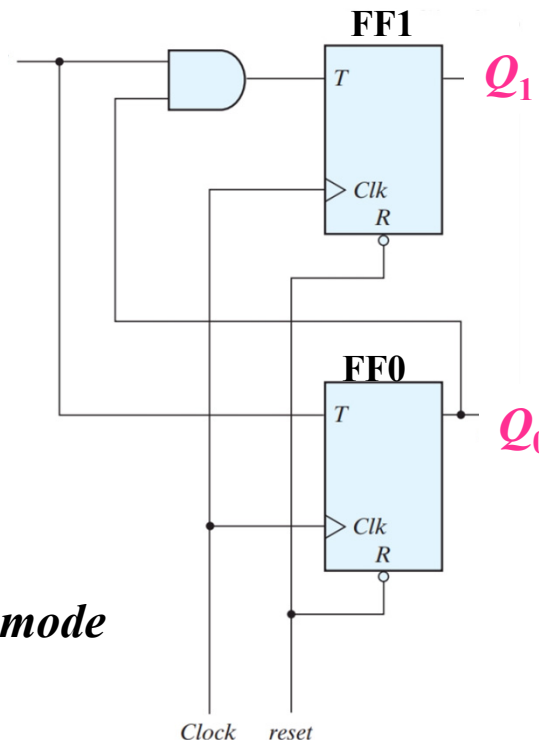


(a) J-K flip-flop

Enable = HIGH

T Flip-Flop		
T	$Q(t + 1) = Q^+$	
0	$Q(t)$	No change
1	$Q'(t)$	Complement

Complement also known as Toggle mode



In all 3 circuits, Q₀ is the LSB

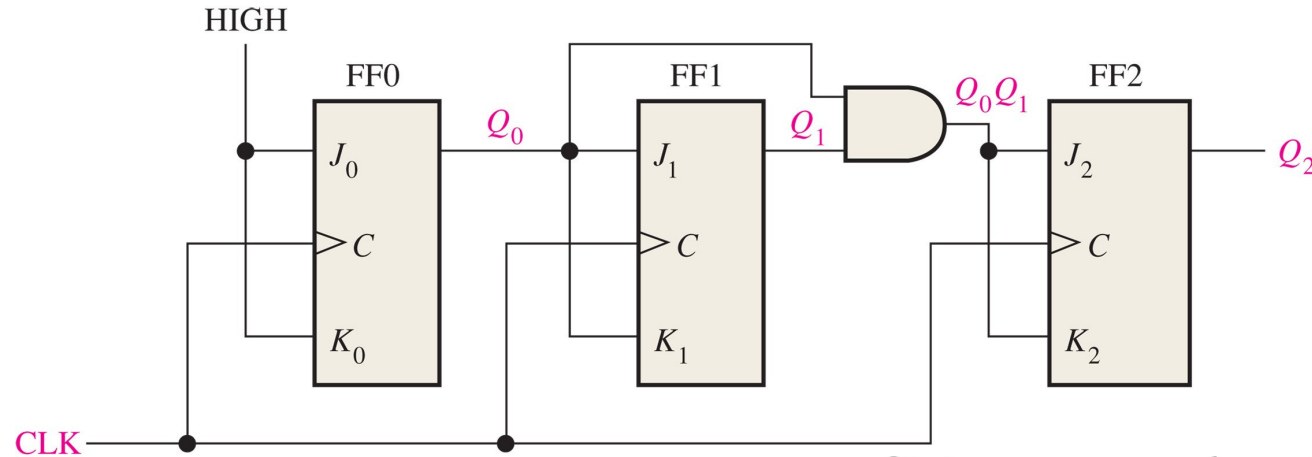
#	Q ₁	Q ₀
0	0	0
1	0	1
2	1	0
3	1	1

Q₀ toggles with every clock edge.

Q₁ toggles if Q₀ is 1 before a clock edge.

3-bit binary synchronous counters

You can replace JK flip-flops with T flip-flops and still works



State sequence for a 3-bit binary counter.

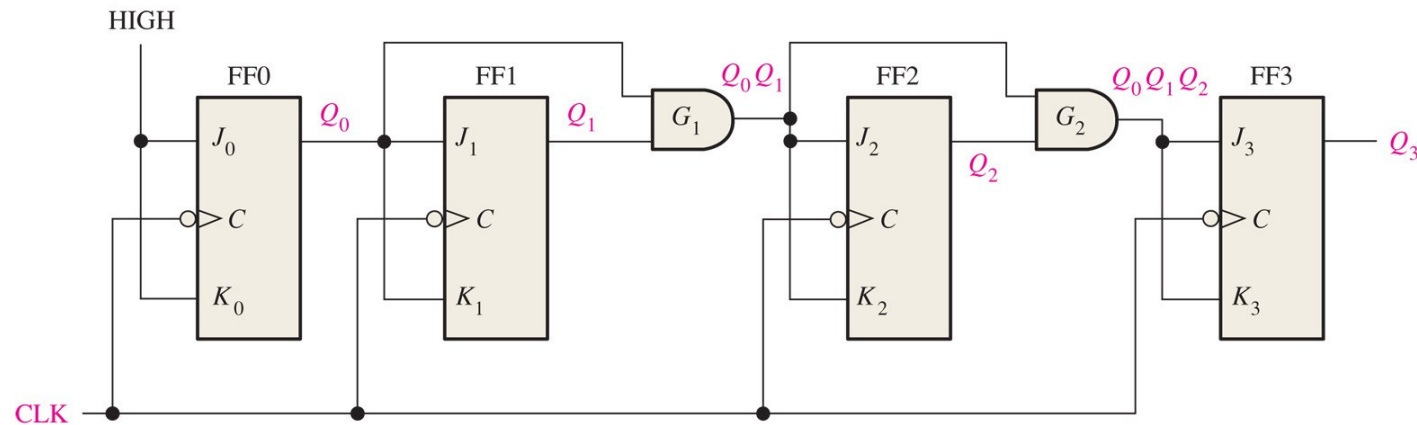
Clock Pulse	Q_2	Q_1	Q_0
Initially	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1
8 (recycles)	0	0	0

Q_0 toggles with every clock edge.

Q_1 toggles if Q_0 is 1 before a clock edge.

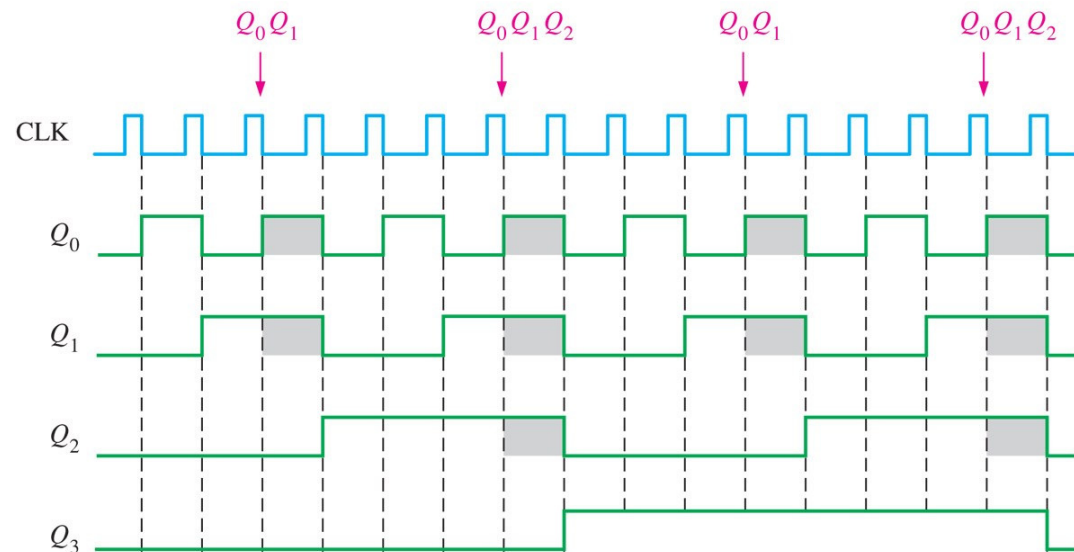
Q_2 toggles if Q_1 AND Q_0 are 1 before a clock edge.

4-bit UP synchronous counter



Toggle bit if all previous bits are '1'

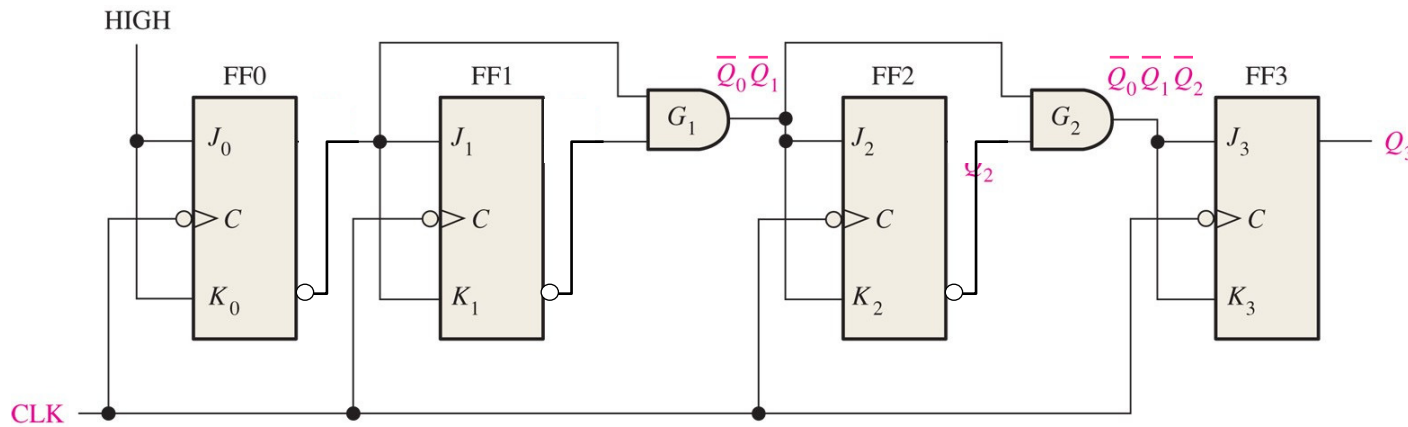
(a) Q_3 toggles if Q_2 AND Q_1 AND Q_0 are 1 before a clock edge.



(b) Times where the AND gate outputs are HIGH are indicated by the shaded areas.

DECIMAL NUMBER	BINARY NUMBER			
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1
12	1	1	0	0
13	1	1	0	1
14	1	1	1	0
15	1	1	1	1

4-bit DOWN synchronous counter



Toggle bit if all previous bits are '0'

Q_0 toggles with every clock edge.

Q_1 toggles if Q_0 is 0 before a clock edge.

Q_2 toggles if Q_1 AND Q_0 are 0 before a clock edge.

Q_3 toggles if Q_2 AND Q_1 AND Q_0 are 0 before a clock edge.

Times where the AND gate outputs are HIGH are indicated by the shaded areas.

DECIMAL NUMBER	BINARY NUMBER			
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1
12	1	1	0	0
13	1	1	0	1
14	1	1	1	0
15	1	1	1	1

3-bit UP/DOWN synchronous counter

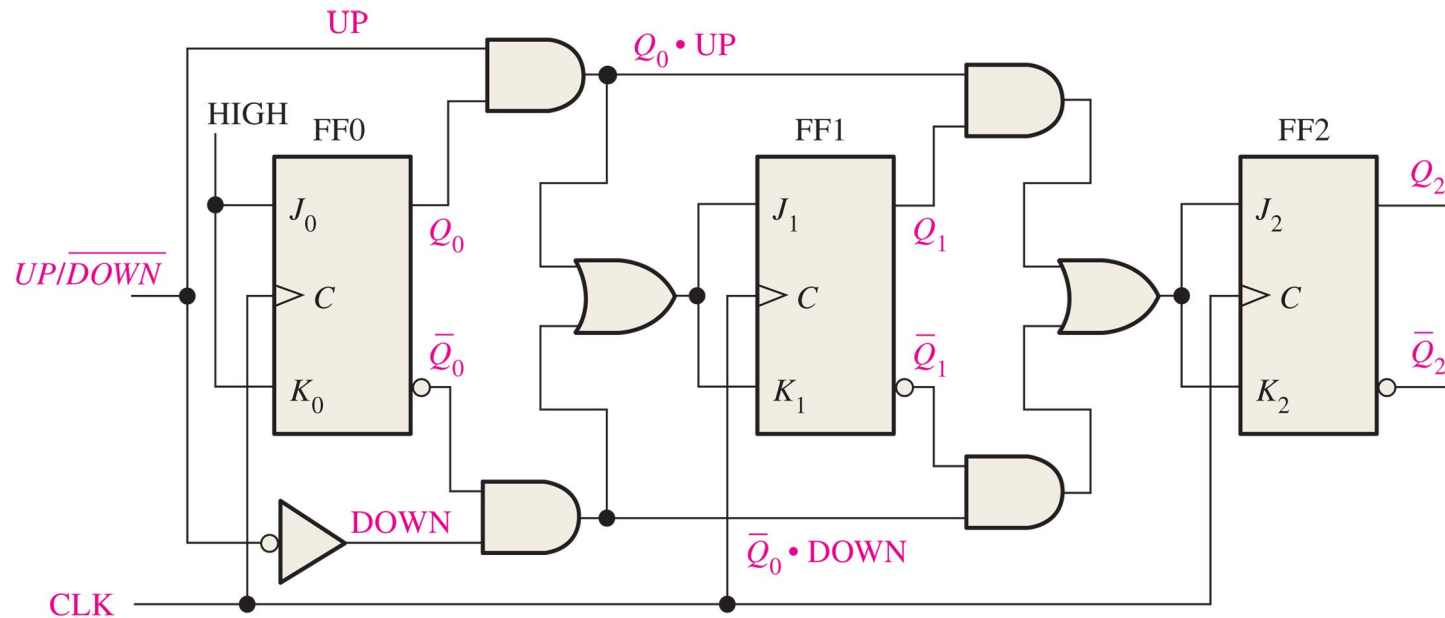
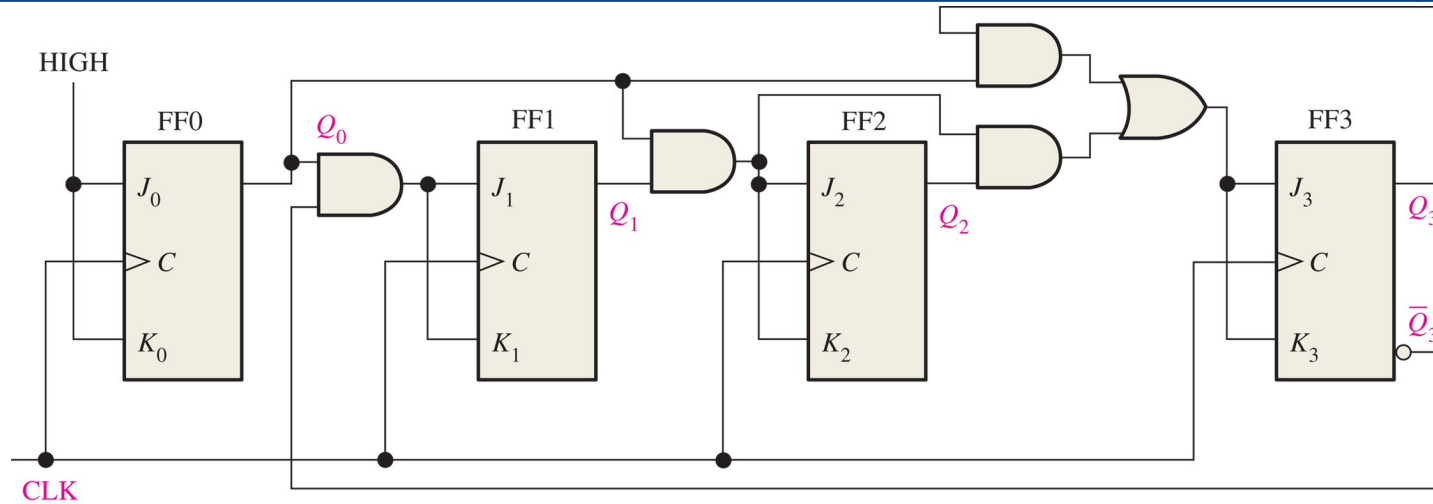


TABLE 9-6

Up/Down sequence for a 3-bit binary counter.

Clock Pulse	Up	Q_2	Q_1	Q_0	Down
0	↶	0	0	0	↷
1	↶	0	0	1	↷
2	↶	0	1	0	↷
3	↶	0	1	1	↷
4	↶	1	0	0	↷
5	↶	1	0	1	↷
6	↶	1	1	0	↷
7	↶	1	1	1	↷

A Synchronous BCD counter



Q_0 toggles with every clock edge.

Q_1 toggles if Q_0 is 1 AND Q_3 is 0 before a clock edge.

Q_2 toggles if Q_1 AND Q_0 are 1 before a clock edge.

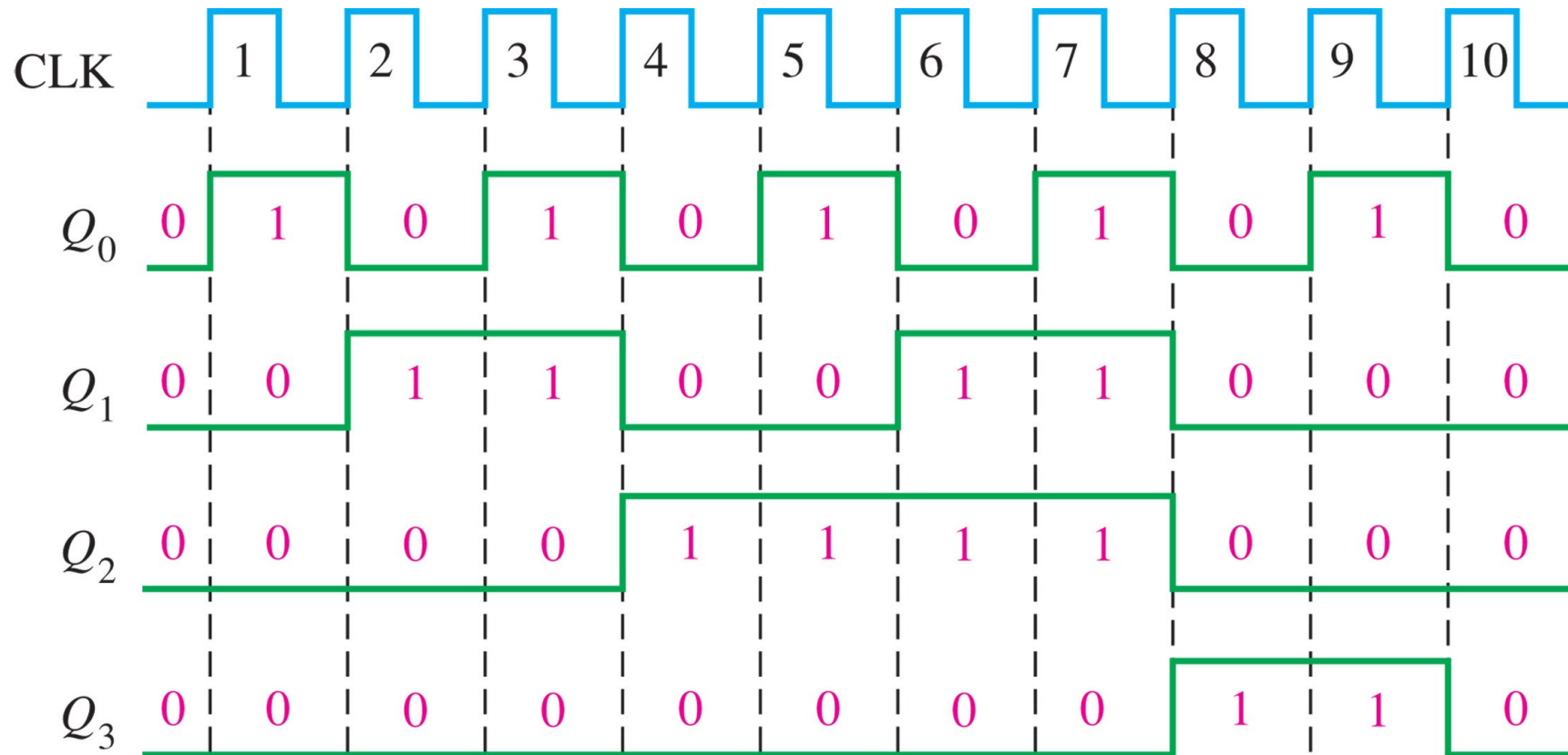
Q_3 toggles if Q_2 AND Q_1 AND Q_0 are 1 OR if Q_3 AND Q_0 are 1 before a clock edge.

Clock Pulse	Q_3	Q_2	Q_1	Q_0
Initially	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10 (recycles)	0	0	0	0

A Synchronous BCD counter

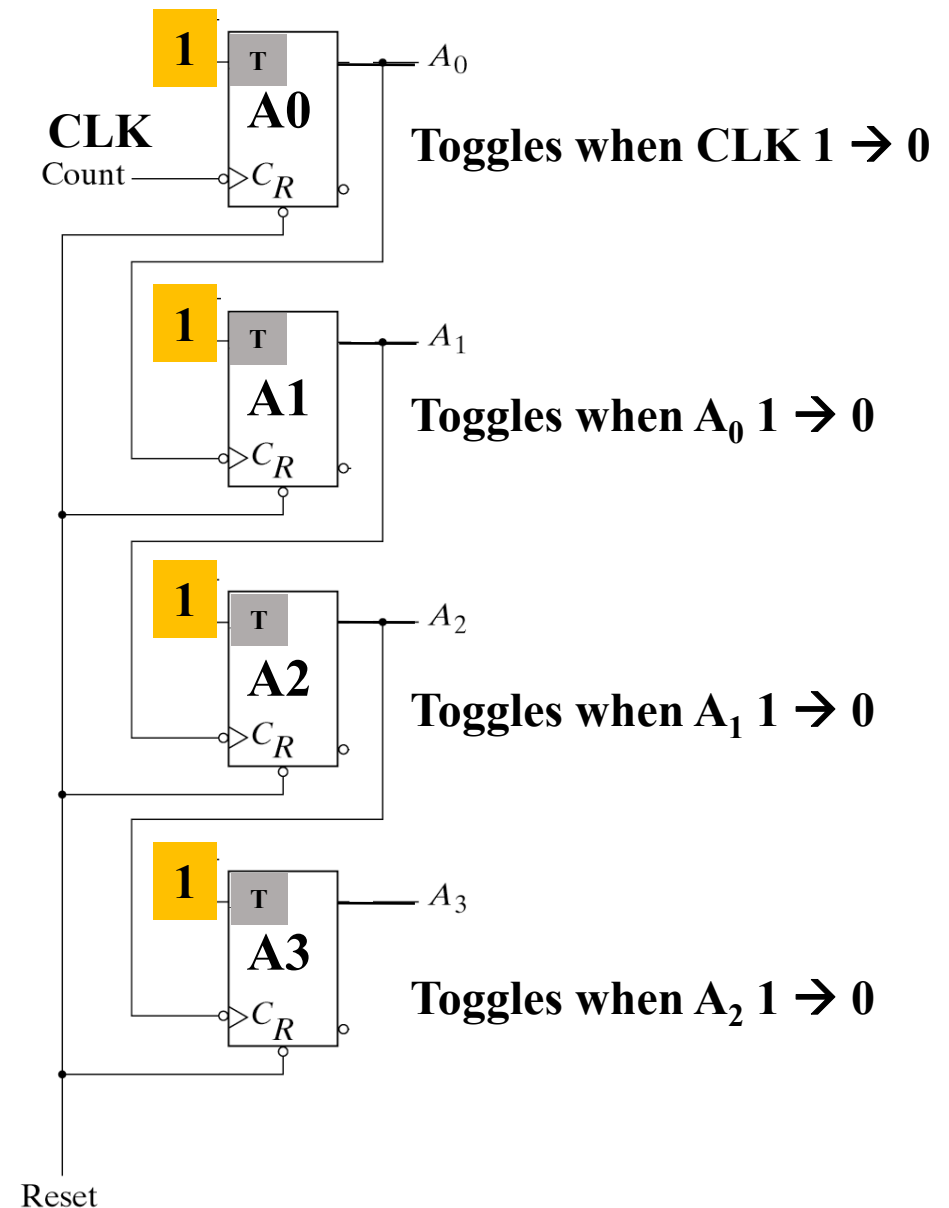


Timing diagram for the BCD decade counter (Q_0 is the LSB).



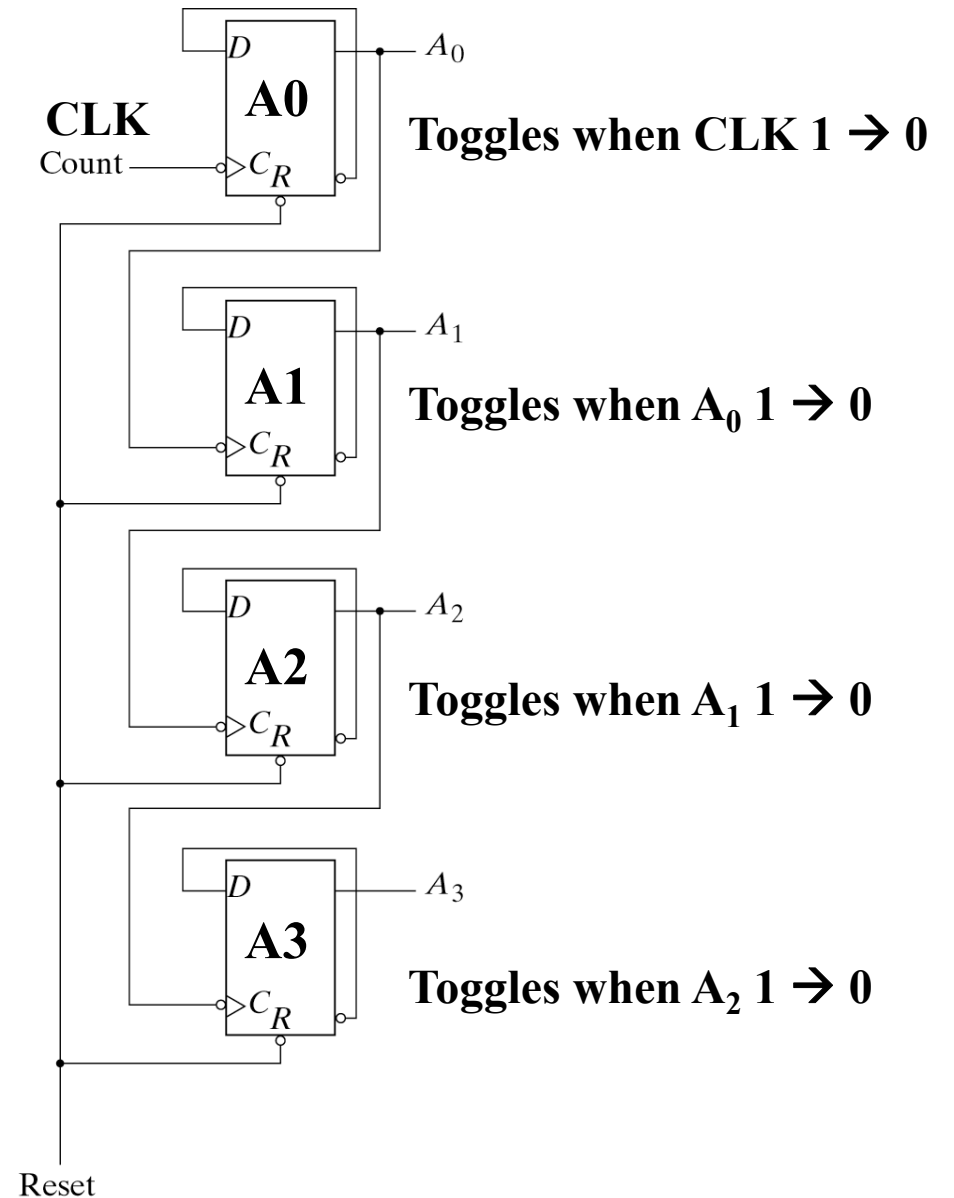
4-bit ripple (asynchronous) UP counter (T-FF)

DECIMAL NUMBER	A3	A2	A1	A0
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1
12	1	1	0	0
13	1	1	0	1
14	1	1	1	0
15	1	1	1	1



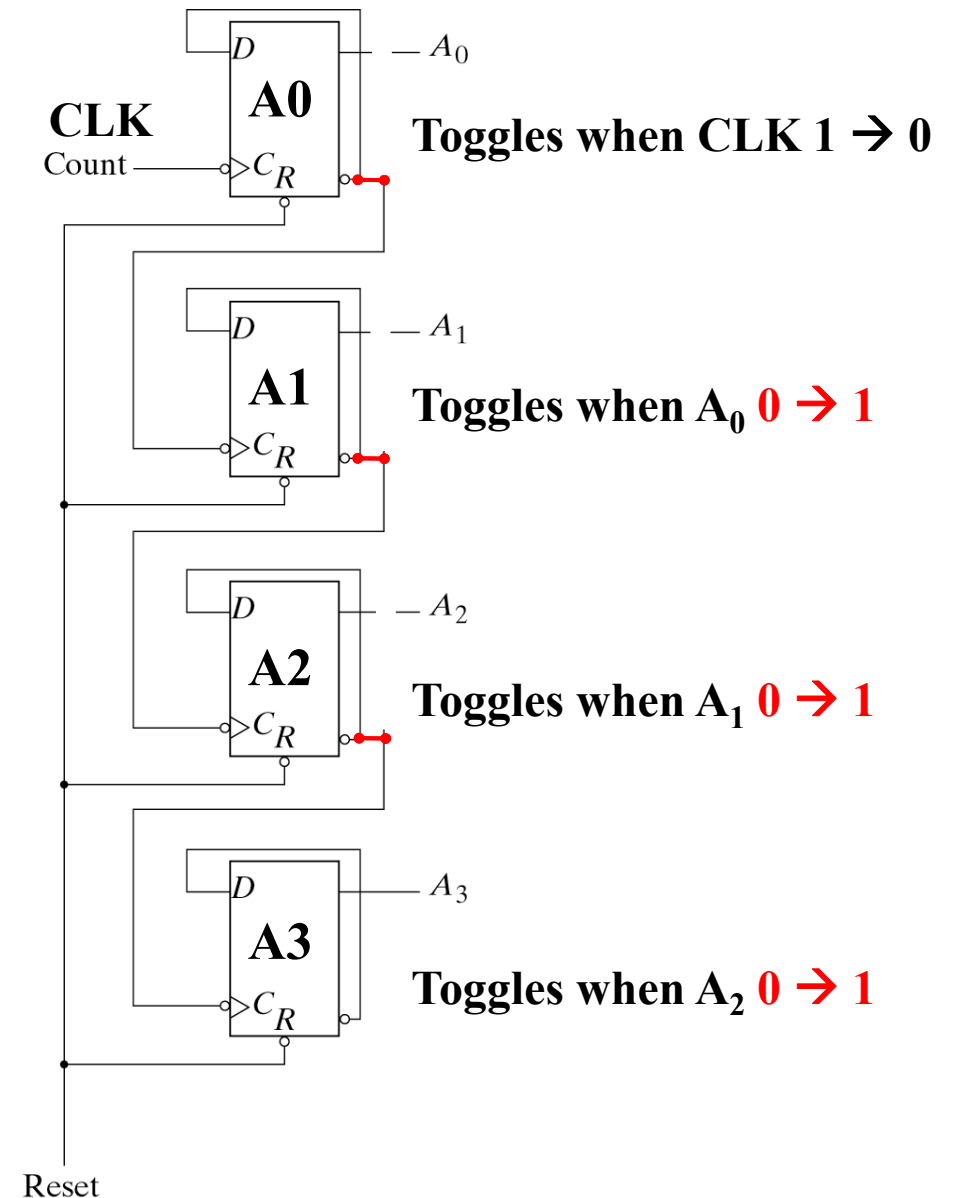
4-bit ripple (asynchronous) UP counter (D-FF)

DECIMAL NUMBER	A3	A2	A1	A0
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1
12	1	1	0	0
13	1	1	0	1
14	1	1	1	0
15	1	1	1	1



4-bit ripple (asynchronous) DOWN counter (D-FF)

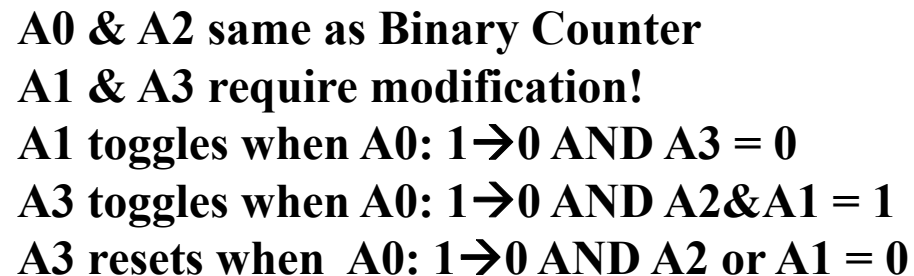
DECIMAL NUMBER	A3	A2	A1	A0
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1
12	1	1	0	0
13	1	1	0	1
14	1	1	1	0
15	1	1	1	1



THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY

[illegible]

THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY



Logic minimization: Karnaugh Map



Karnaugh Map: Truth table in 2-dimensional layout

A 3-variable K-map

		C	
		0	1
Gray Code	AB		
	00	$\bar{A}\bar{B}\bar{C}$	$\bar{A}\bar{B}C$
	01	$\bar{A}B\bar{C}$	$\bar{A}BC$
	11	$AB\bar{C}$	ABC
	10	$A\bar{B}\bar{C}$	$A\bar{B}C$

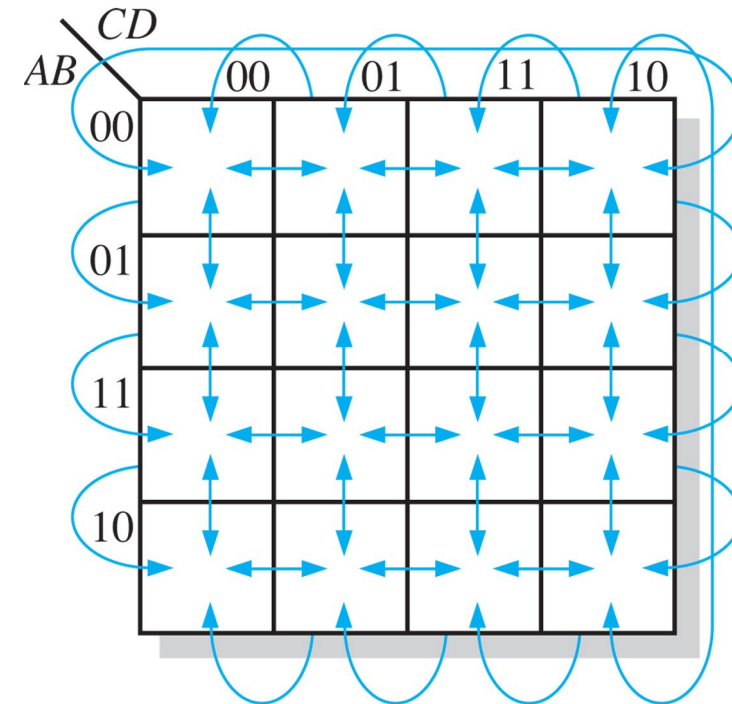
A 4-variable K-map

		Gray Code			
		00	01	11	10
Gray Code	AB				
	00	$\bar{A}\bar{B}\bar{C}\bar{D}$	$\bar{A}\bar{B}\bar{C}D$	$\bar{A}\bar{B}CD$	$\bar{A}\bar{B}C\bar{D}$
	01	$\bar{A}\bar{B}\bar{C}\bar{D}$	$\bar{A}\bar{B}\bar{C}D$	$\bar{A}\bar{B}CD$	$\bar{A}\bar{B}C\bar{D}$
	11	$AB\bar{C}\bar{D}$	$AB\bar{C}D$	$ABCD$	$ABC\bar{D}$
	10	$A\bar{B}\bar{C}\bar{D}$	$A\bar{B}\bar{C}D$	$A\bar{B}CD$	$A\bar{B}C\bar{D}$

Logic minimization: Karnaugh Map

Adjacent cells on a K-map are those that differ by only one variable. Arrows point between adjacent cells.

AB \ CD	CD			
	00	01	11	10
00	$\bar{A}\bar{B}\bar{C}\bar{D}$	$\bar{A}\bar{B}\bar{C}D$	$\bar{A}\bar{B}C\bar{D}$	$\bar{A}\bar{B}CD$
01	$\bar{A}B\bar{C}\bar{D}$	$\bar{A}B\bar{C}D$	$\bar{A}BC\bar{D}$	$\bar{A}BCD$
11	$AB\bar{C}\bar{D}$	$AB\bar{C}D$	$ABCD$	$ABC\bar{D}$
10	$A\bar{B}\bar{C}\bar{D}$	$A\bar{B}\bar{C}D$	$A\bar{B}C\bar{D}$	$A\bar{B}CD$



Logic minimization: Karnaugh Map



Example of mapping directly from a truth table to a Karnaugh map.

$$X = \bar{A}\bar{B}\bar{C} + A\bar{B}\bar{C} + AB\bar{C} + ABC$$

Inputs			Output		
<i>A</i>	<i>B</i>	<i>C</i>	<i>X</i>		
0	0	0	1	00	1
0	0	1	0	01	0
0	1	0	0		
0	1	1	0	11	1
1	0	0	1		
1	0	1	0	10	1
1	1	0	1		
1	1	1	1		

<i>AB</i> \ <i>C</i>		0	1
		0	1
00	1	0	
01	0	0	
11	1	1	
10	1	0	

Logic minimization starts with grouping the 1s on the K-map by enclosing those adjacent cells containing 1s.

The goal is to maximize the size of the groups and to minimize the number of groups.

You can group 1s according to the following rules

- 1. A group must contain either 1, 2, 4, 8, or 16 cells, which are all powers of two. In the case of a 3-variable map, $2^3 = 8$ cells is the maximum group.**
- 2. Each cell in a group must be adjacent to one or more cells in that same group, but all cells in the group do not have to be adjacent to each other. Note diagonal cells are NOT adjacent.**
- 3. Always include the largest possible number of 1s in a group in accordance with rule 1.**
- 4. Each 1 on the map must be included in at least one group. The 1s already in a group can be included in another group as long as the overlapping groups include noncommon 1s.**

The following rules are applied to find the minimum product terms and the minimum SOP expression:

1. Group the cells that have 1s. Each group of cells containing 1s creates one product term composed of all variables that occur in only one form (either uncomplemented or complemented) within the group. **Variables that occur both uncomplemented and complemented within the group are eliminated.** These are called contradictory variables.
2. Determine the minimum product term for each group.
 - See Next Slide
3. When all the minimum product terms are derived from the K-map, they are summed to form the minimum SOP expression.

2. Determine the minimum product term for each group

a) For a 3-variable map:

1. A 1-cell group yields a 3-variable product term
2. A 2-cell group yields a 2-variable product term
3. A 4-cell group yields a 1-variable term
4. An 8-cell group yields a value of 1 for the expression

b) For a 4-variable map:

1. A 1-cell group yields a 4-variable product term
2. A 2-cell group yields a 3-variable product term
3. A 4-cell group yields a 2-variable product term
4. An 8-cell group yields a 1-variable term
5. A 16-cell group yields a value of 1 for the expression

AB \ C	0	1
00	$\bar{A}\bar{B}\bar{C}$	$\bar{A}\bar{B}C$
01	$\bar{A}B\bar{C}$	$\bar{A}BC$
11	$AB\bar{C}$	ABC
10	$A\bar{B}\bar{C}$	$A\bar{B}C$

AB \ CD	00	01	11	10
00	$\bar{A}\bar{B}\bar{C}\bar{D}$	$\bar{A}\bar{B}\bar{C}D$	$\bar{A}\bar{B}C\bar{D}$	$\bar{A}\bar{B}CD$
01	$\bar{A}B\bar{C}\bar{D}$	$\bar{A}B\bar{C}D$	$\bar{A}BC\bar{D}$	$\bar{A}BCD$
11	$AB\bar{C}\bar{D}$	$AB\bar{C}D$	$ABC\bar{D}$	$ABCD$
10	$A\bar{B}\bar{C}\bar{D}$	$A\bar{B}\bar{C}D$	$A\bar{B}C\bar{D}$	$A\bar{B}CD$

Logic minimization: Karnaugh Map

EXAMPLE: Determine the product terms for the Karnaugh map and write the resulting minimum SOP expression

$AB \backslash CD$					
		00	01	11	10
00				1	1
01	1	1	1	1	1
11	1	1	1	1	1
10			1		

Logic minimization: Karnaugh Map

EXAMPLE: Determine the product terms for the Karnaugh map and write the resulting minimum SOP expression.

AB \ C	0	1
00	1	
01		1
11	1	1
10		

AB \ C	0	1
00	1	1
01	1	
11		1
10	1	1

AB \ CD	00	01	11	10
00	1	1		
01	1	1	1	1
11				
10		1	1	

AB \ CD	00	01	11	10
00	1			1
01	1	1		1
11	1	1		1
10	1		1	1

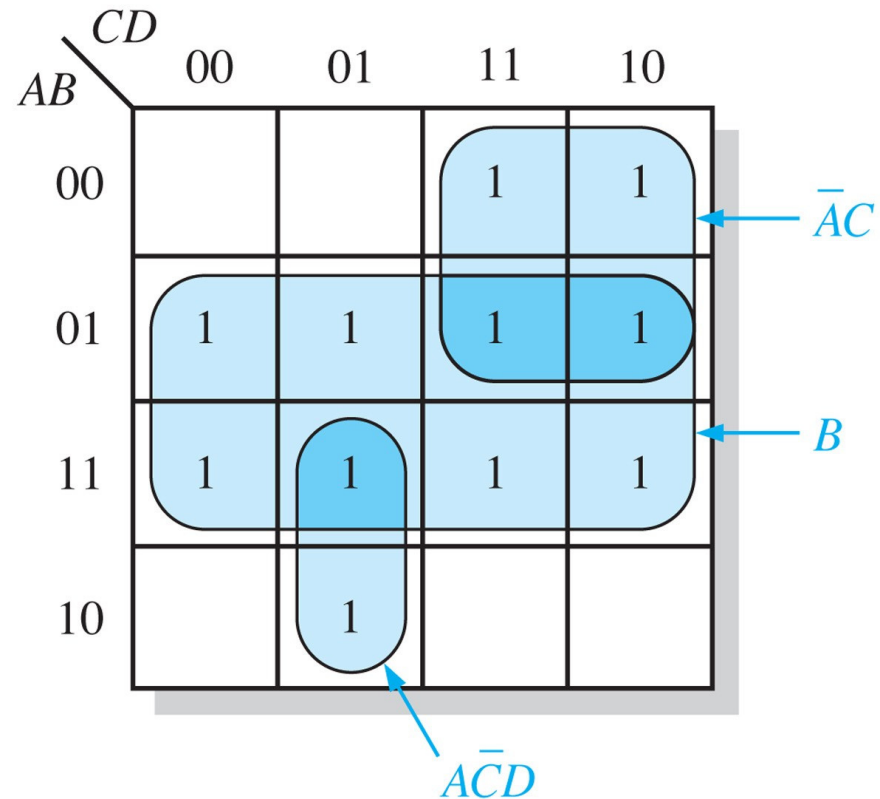
EXAMPLE: Determine the product terms for the Karnaugh map and write the resulting minimum SOP expression.

AB \ C	C	
	0	1
00	1	1
01		1
11		
10	1	1

AB \ CD	CD			
	00	01	11	10
00	1		1	1
01	1			1
11	1			1
10	1		1	1

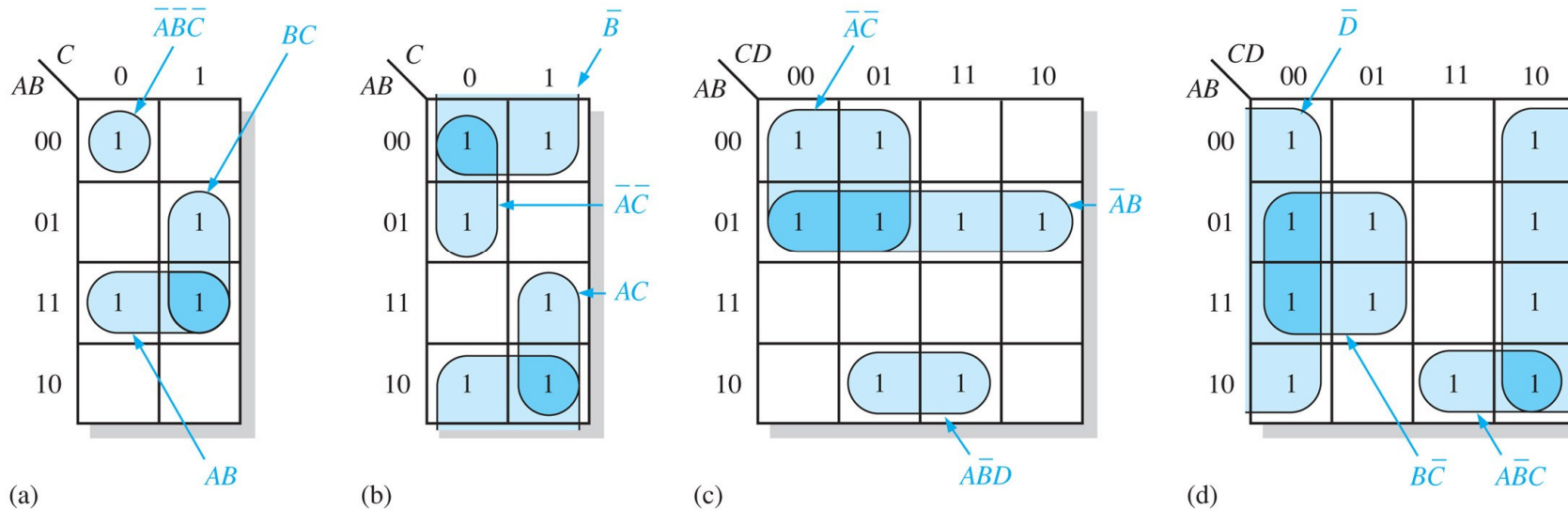
Logic minimization: Karnaugh Map

EXAMPLE: Determine the product terms for the Karnaugh map and write the resulting minimum SOP expression.



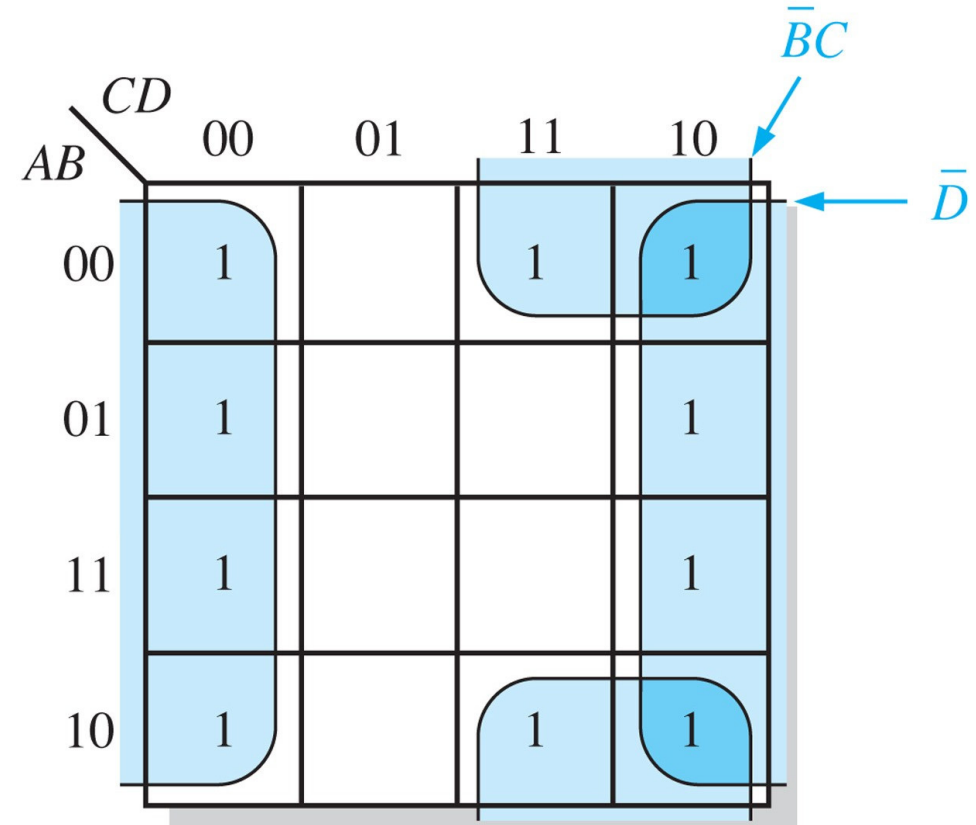
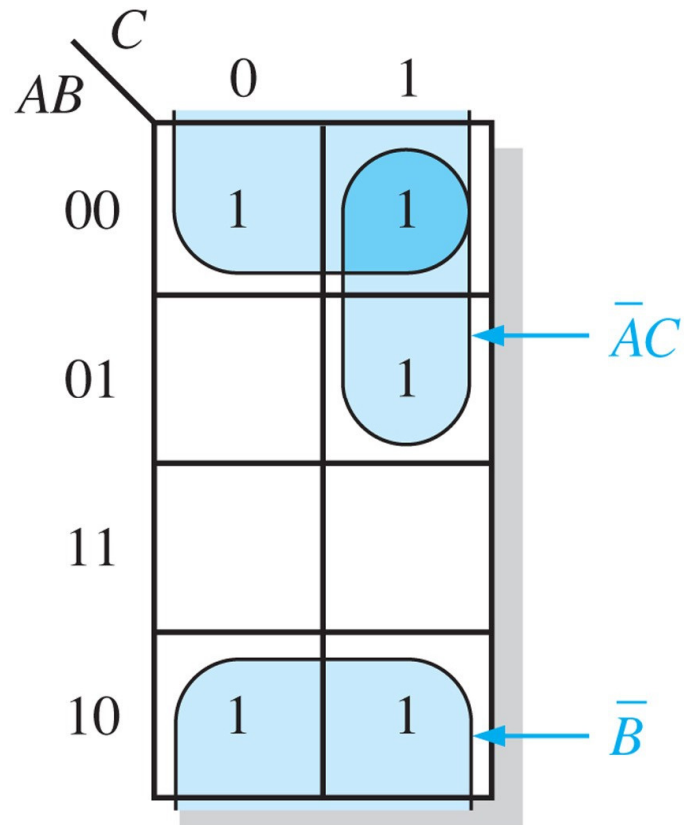
Logic minimization: Karnaugh Map

EXAMPLE: Determine the product terms for the Karnaugh map and write the resulting minimum SOP expression.



Logic minimization: Karnaugh Map

EXAMPLE: Determine the product terms for the Karnaugh map and write the resulting minimum SOP expression.



Logic minimization: Karnaugh Map



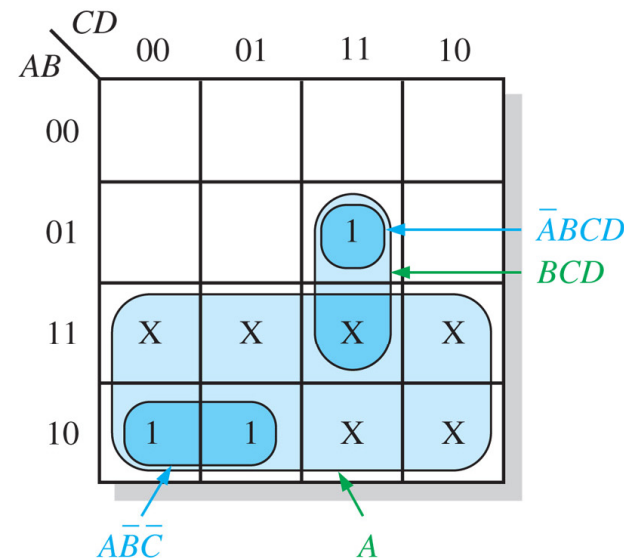
Example of the use of “don’t care” conditions to simplify an expression.

Inputs				Output
A	B	C	D	Y
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	X
1	0	1	1	X
1	1	0	0	X
1	1	0	1	X
1	1	1	0	X
1	1	1	1	X

(a) Truth table

- “X” means “don’t care”
- “X” can be either 0 or 1
- It is up to you to assume whichever of X’s zero or 1

Don’t cares



All X’s are assumed 1 here
because it helps to further
minimize the expression
and the logic circuit

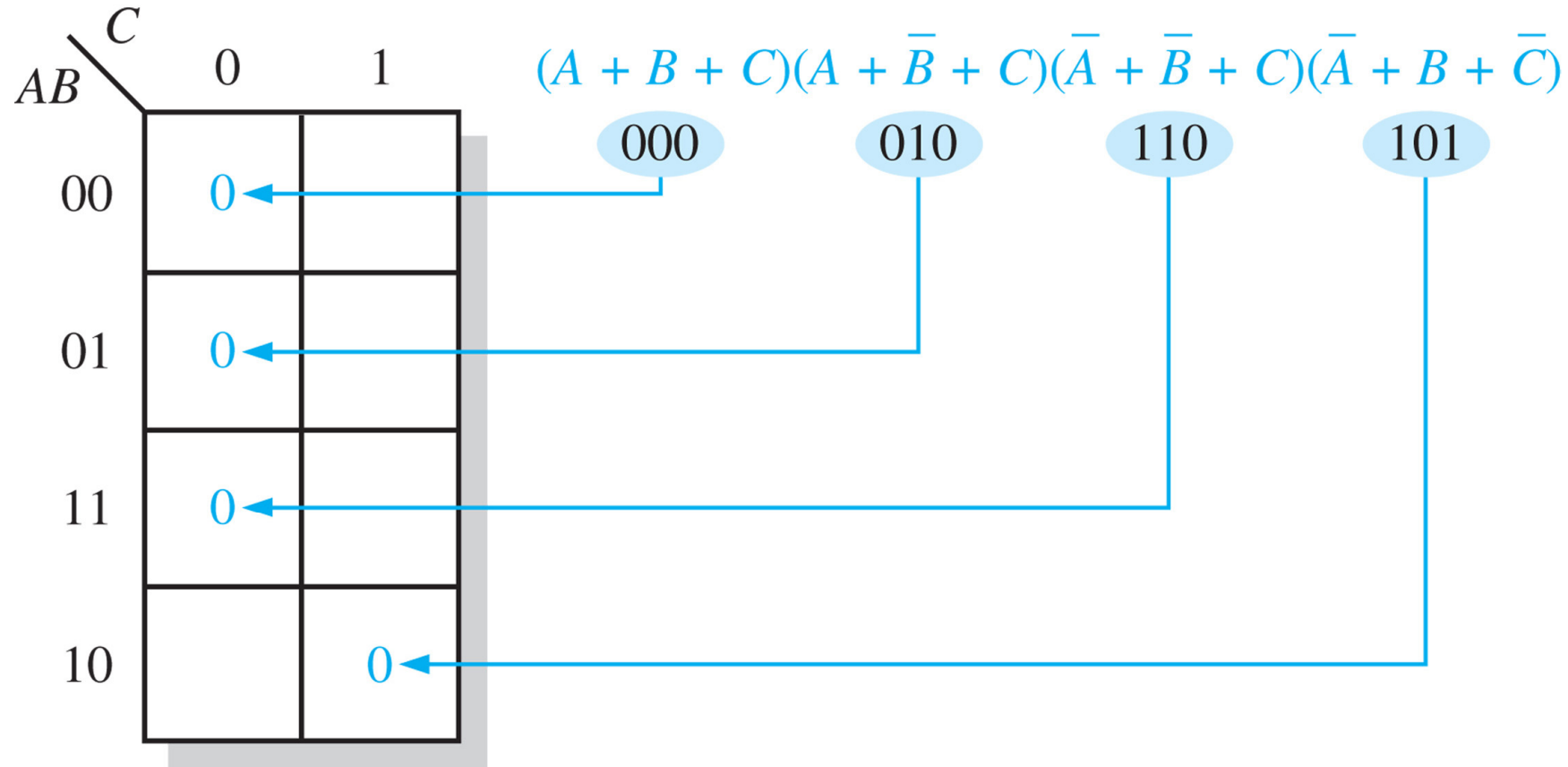
(b) Without “don’t cares” $Y = A\bar{B}\bar{C} + \bar{A}BCD$
With “don’t cares” $Y = A + BCD$

Logic minimization: Karnaugh Map



THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY

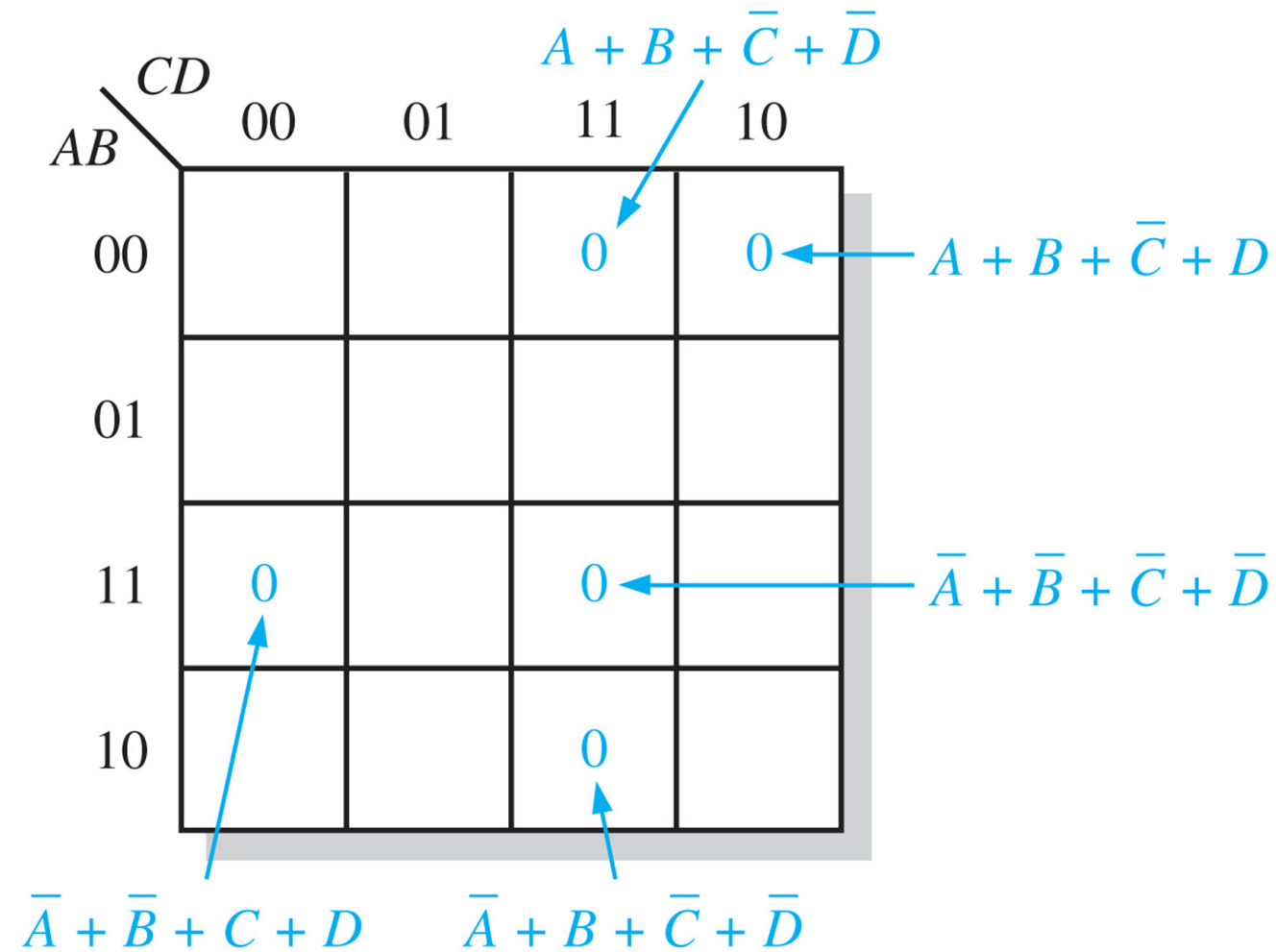
Same rules apply to Product of Sums (POS) but you have to work with 0s and MAXTERMS (Sum terms)



Logic minimization: Karnaugh Map



THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY



Logic minimization: Karnaugh Map

Rules for grouping are the same except each group represents a **SUM** term (not a **PRODUCT**)

		C	
		0	1
AB	00	0	0
	01	0	0
	11	0	1
	10	1	1

Logic minimization: Karnaugh Map

Rules for grouping are the same except each group represents a **SUM** term (not a **PRODUCT**)

$AB \backslash CD$					
		00	01	11	10
00		0			0
01		0			
11		0			
10		0	0		

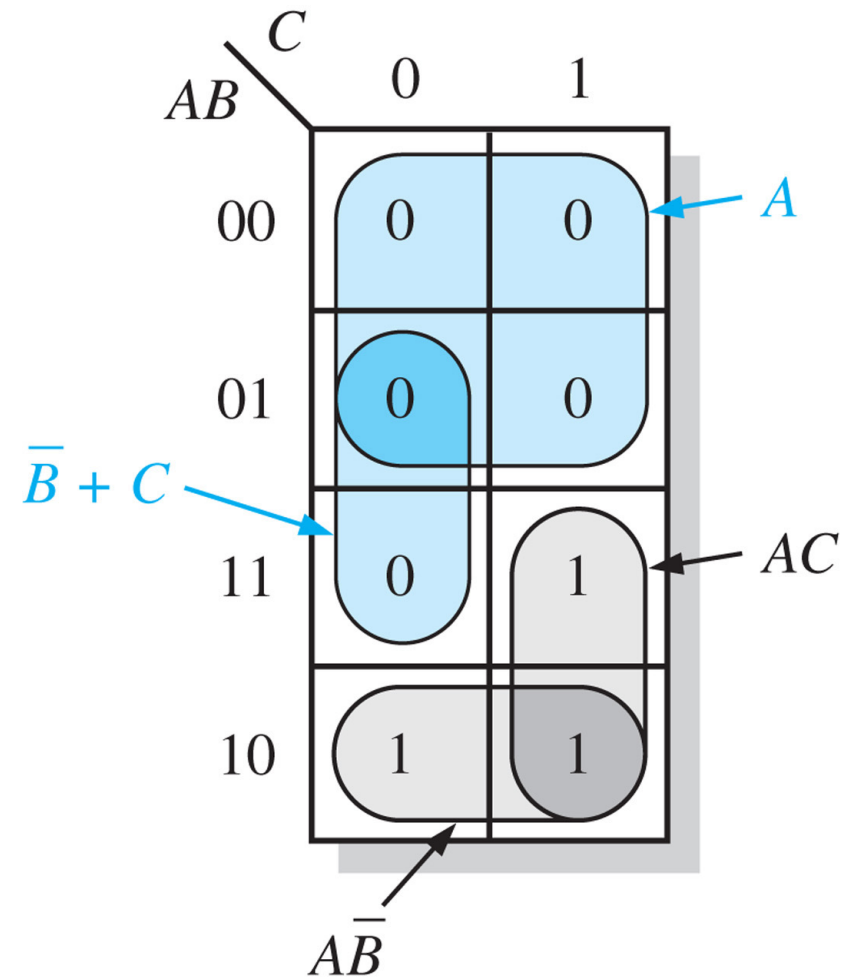
Logic minimization: Karnaugh Map

Rules for grouping are the same except each group represents a **SUM** term (not a **PRODUCT**)

$AB \backslash CD$					
		00	01	11	10
00			0	0	0
01		0			
11		0			
10			0		

Logic minimization: Karnaugh Map

Rules for grouping are the same except each group represents a **SUM** term (not a **PRODUCT**)

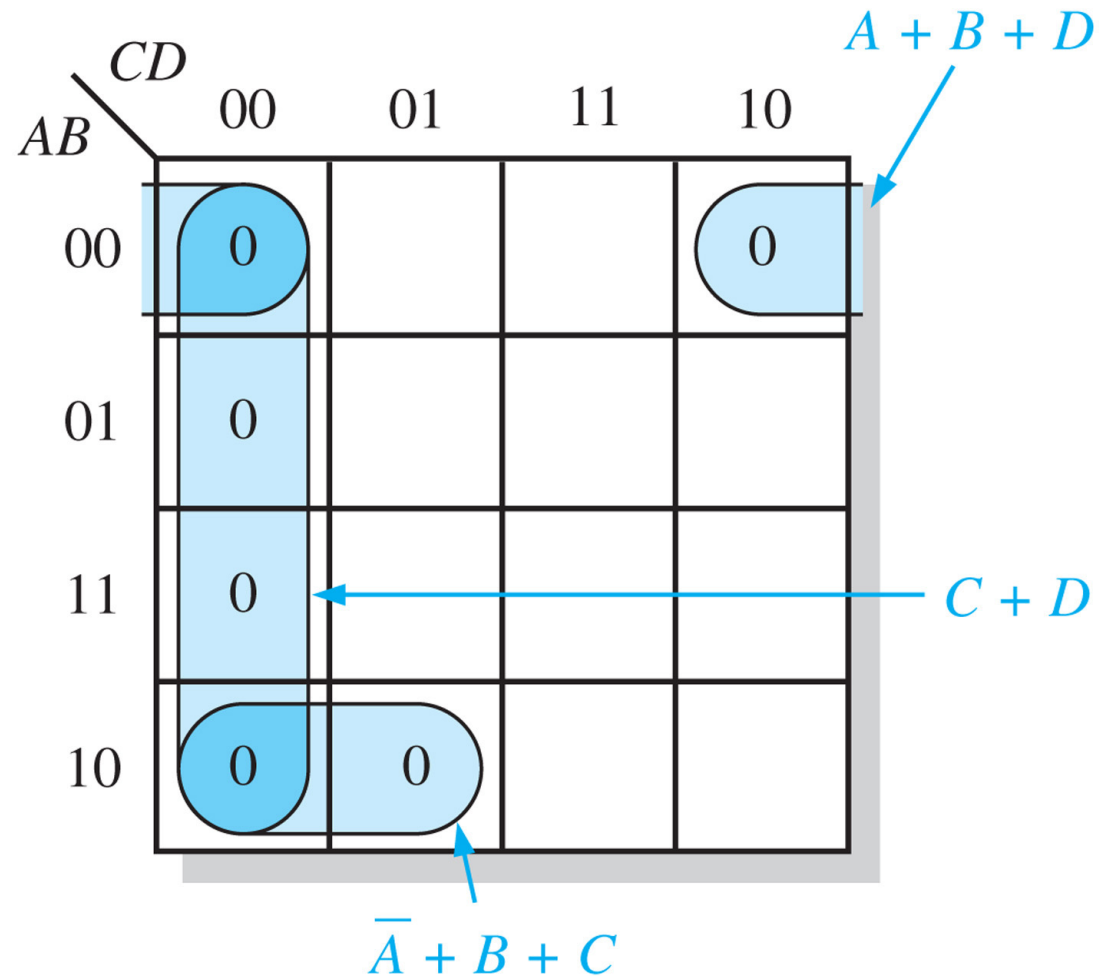


Logic minimization: Karnaugh Map



THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY

Rules for grouping are the same except each group represents a **SUM** term (not a **PRODUCT**)

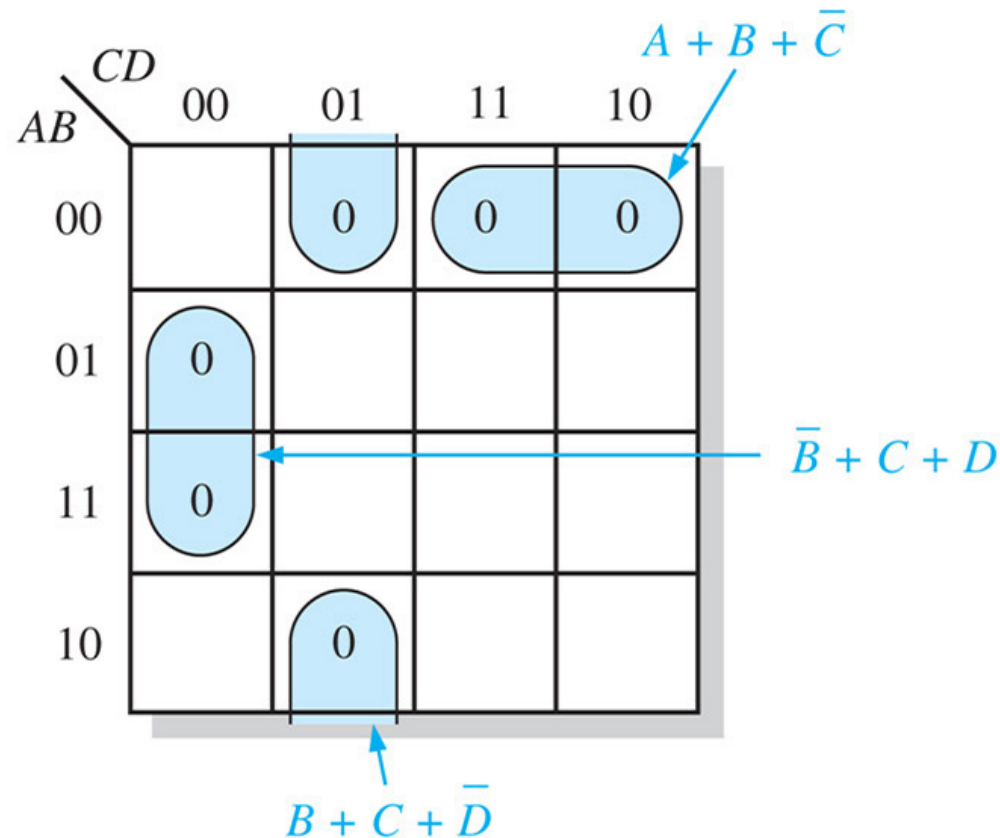


Logic minimization: Karnaugh Map



THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY

Rules for grouping are the same except each group represents a SUM term (not a PRODUCT)



(a) Minimum POS: $(A + B + C)(\bar{B} + \bar{C} + D)(B + C + \bar{D})$

3-bit Gray Code Sync. counter



State diagram

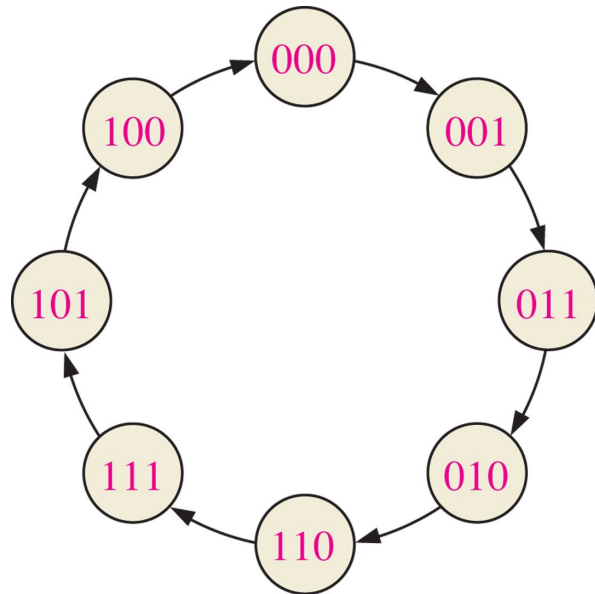


TABLE 9-8

Next-state table for 3-bit Gray code counter.

Present State			Next State		
Q_2	Q_1	Q_0	Q_2	Q_1	Q_0
0	0	0	0	0	1
0	0	1	0	1	1
0	1	1	0	1	0
0	1	0	1	1	0
1	1	0	1	1	1
1	1	1	1	0	1
1	0	1	1	0	0
1	0	0	0	0	0

1. Obtain specification of the desired counter
2. Create a state diagram from specification.
3. Create a state table from state diagram.
4. Choose flip-flop types (D, JK, or T)
5. Derive the next-state logic and output logic (if any) expressions.
6. Implement circuit described by expressions.

4. Choose flip-flop types (D, JK, or T)

Let's choose JK flip flops for all 3 bits

Characteristic Table for JK Flip Flop

JK Flip-Flop			
<i>J</i>	<i>K</i>	<i>Q(t + 1)</i>	
0	0	<i>Q(t)</i>	No change
0	1	0	Reset
1	0	1	Set
1	1	<i>Q'(t)</i>	Complement

“X” means “don’t care”
“X” can be 0 or 1
Because both 0 and 1 work

TABLE 9-9

Transition table for a J-K flip-flop.

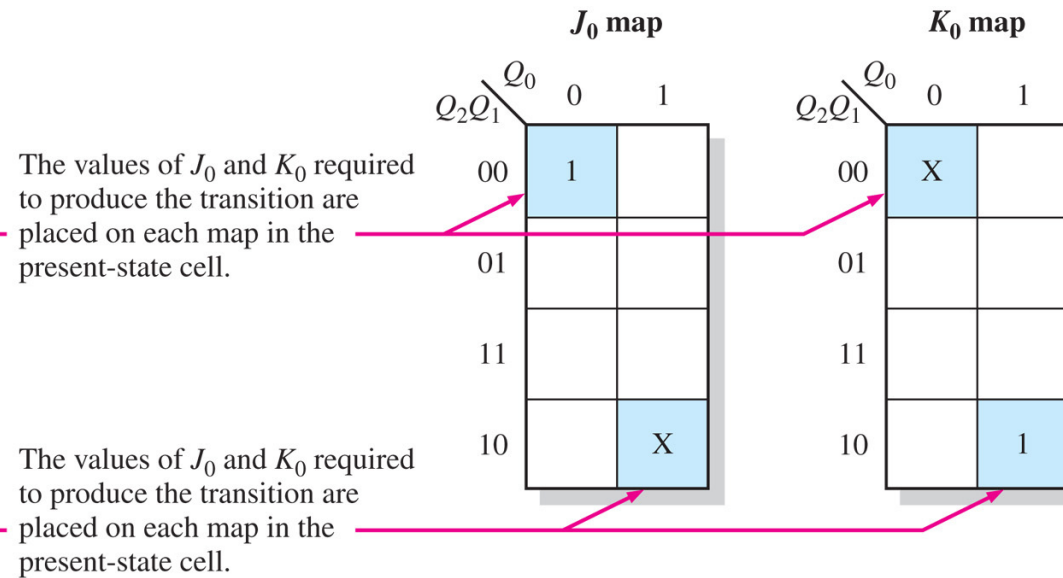
Output Transitions			Flip-Flop Inputs	
Q_N		Q_{N+1}	<i>J</i>	<i>K</i>
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : present state

Q_{N+1} : next state

X: “don’t care”

3-bit Gray Code counter



Output Transitions		Flip-Flop Inputs	
Q_N	Q_{N+1}	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

Flip-flop transition table

Present State			Next State		
Q_2	Q_1	Q_0	Q_2	Q_1	Q_0
0	0	0	0	0	1
0	0	1	0	1	1
0	1	1	0	1	0
0	1	0	1	1	0
1	1	0	1	1	1
1	1	1	1	0	1
1	0	1	1	0	0
1	0	0	0	0	0

Next-state table

For the present state 000, Q_0 makes a transition from 0 to 1 to the next state.

For the present state 101, Q_0 makes a transition from 1 to 0 to the next state.

3-bit Gray Code counter



Karnaugh maps for present-state J and K inputs.

Q_2Q_1	Q_0	
	0	1
00		
01		
11		
10		

$J_2 = ?$

Q_2Q_1	Q_0	
	0	1
00		
01		
11		
10		

$J_1 = ?$

Q_2Q_1	Q_0	
	0	1
00		
01		
11		
10		

$J_0 = ?$

Q_2Q_1	Q_0	
	0	1
00		
01		
11		
10		

$K_2 = ?$

Q_2Q_1	Q_0	
	0	1
00		
01		
11		
10		

$K_1 = ?$

Q_2Q_1	Q_0	
	0	1
00		
01		
11		
10		

$K_0 = ?$

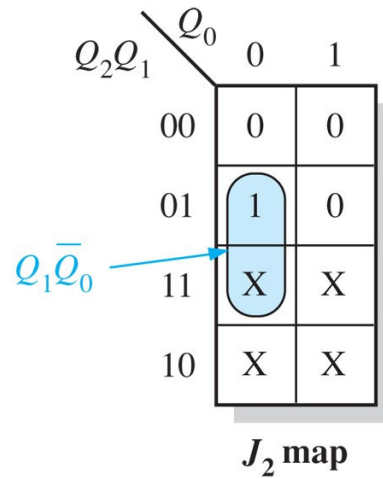
3-bit Gray Code counter

Implement circuit described by expressions

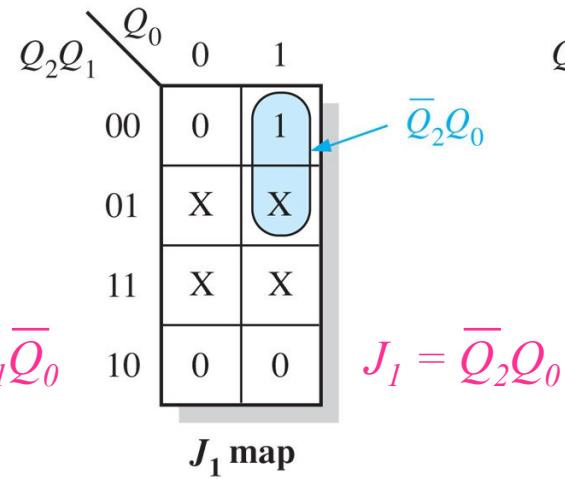
3-bit Gray Code counter



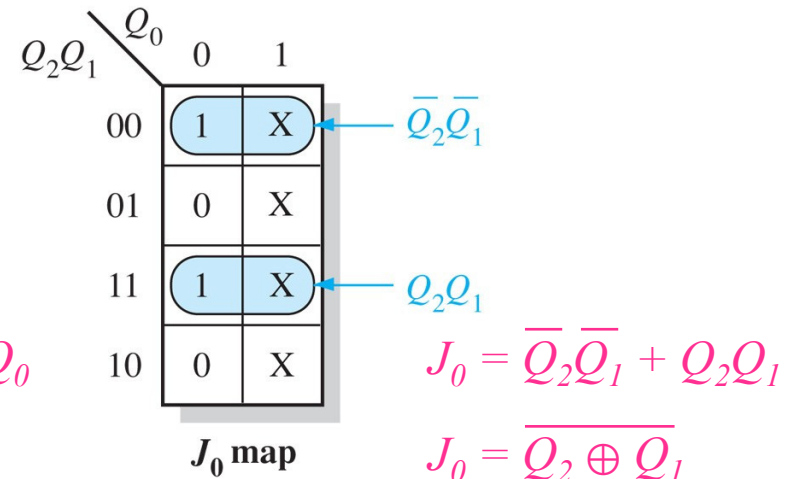
Karnaugh maps for present-state J and K inputs



$$J_2 = Q_1\bar{Q}_0$$



$$J_1 = \bar{Q}_2Q_0$$

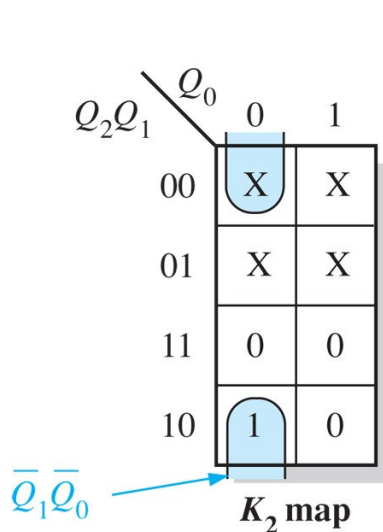


$$J_0 = \bar{Q}_2\bar{Q}_1 + Q_2Q_1$$

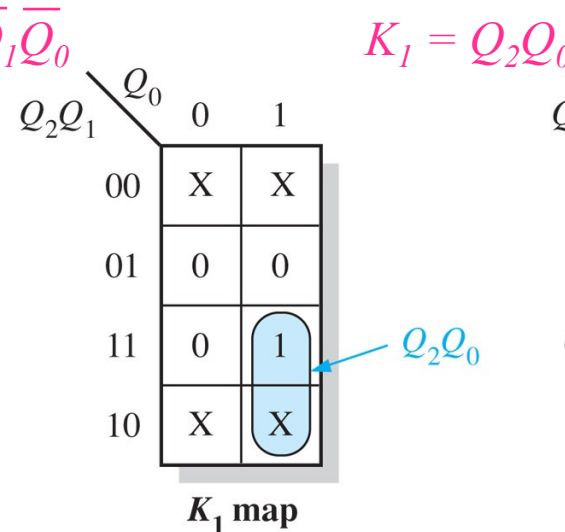
$$J_0 = \bar{Q}_2 \oplus \bar{Q}_1$$

$$K_0 = \bar{Q}_2Q_1 + Q_2\bar{Q}_1$$

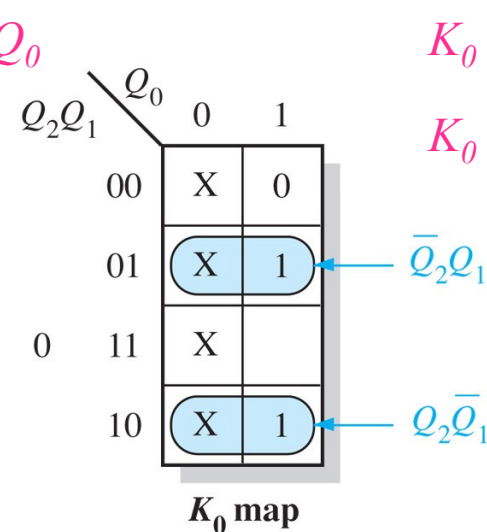
$$K_0 = Q_2 \oplus Q_1$$



$$K_2 = \bar{Q}_1\bar{Q}_0$$



$$K_1 = Q_2Q_0$$



$$J_0 = Q_2Q_1 + \bar{Q}_2\bar{Q}_1 = \bar{Q}_2 \oplus \bar{Q}_1$$

$$K_0 = Q_2\bar{Q}_1 + \bar{Q}_2Q_1 = Q_2 \oplus Q_1$$

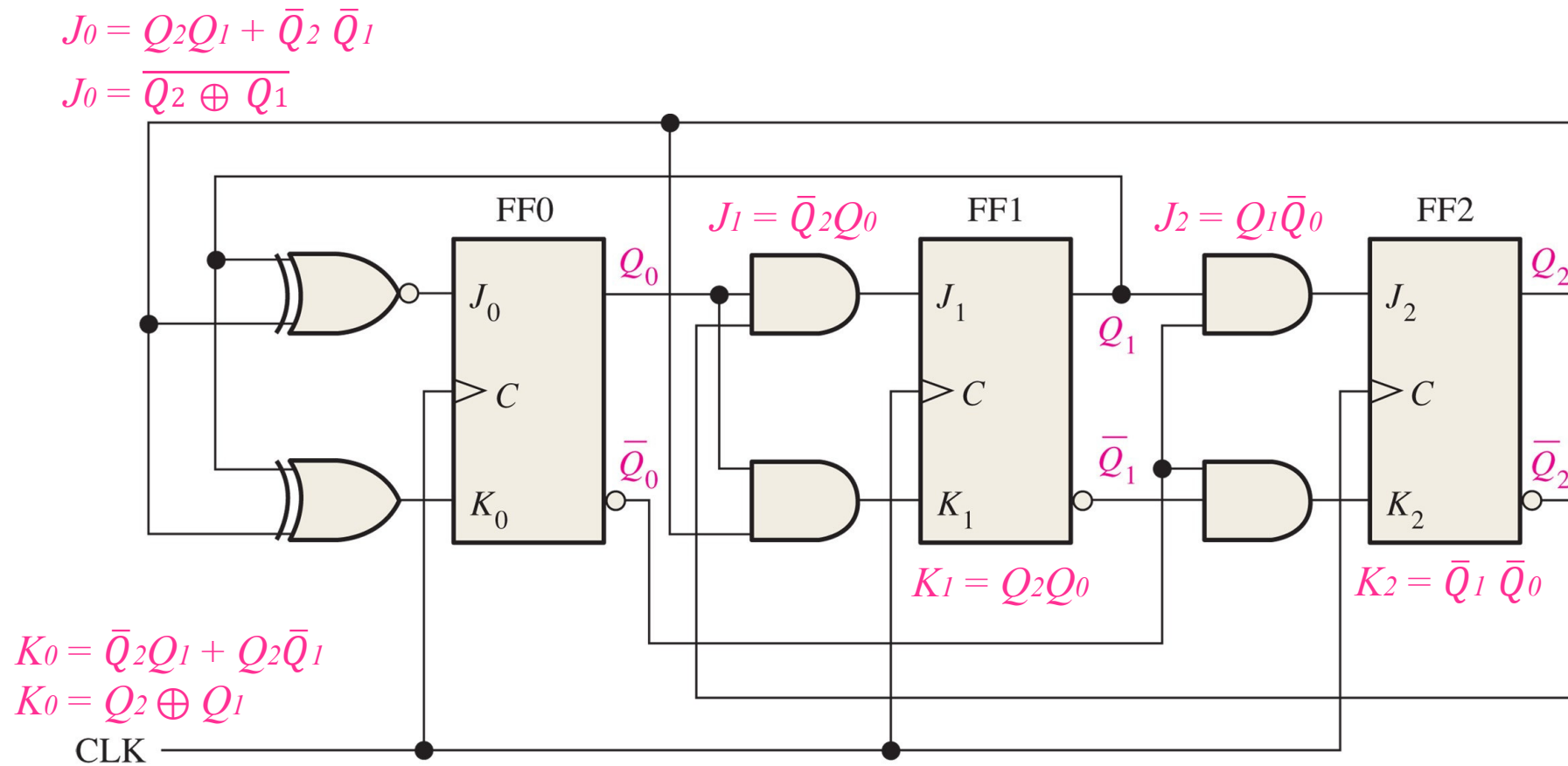
$$J_1 = \bar{Q}_2Q_0$$

$$K_1 = Q_2Q_0$$

$$J_2 = Q_1\bar{Q}_0$$

$$K_2 = \bar{Q}_1\bar{Q}_0$$

3-bit Gray Code counter



Special Counter using JK flip-flops

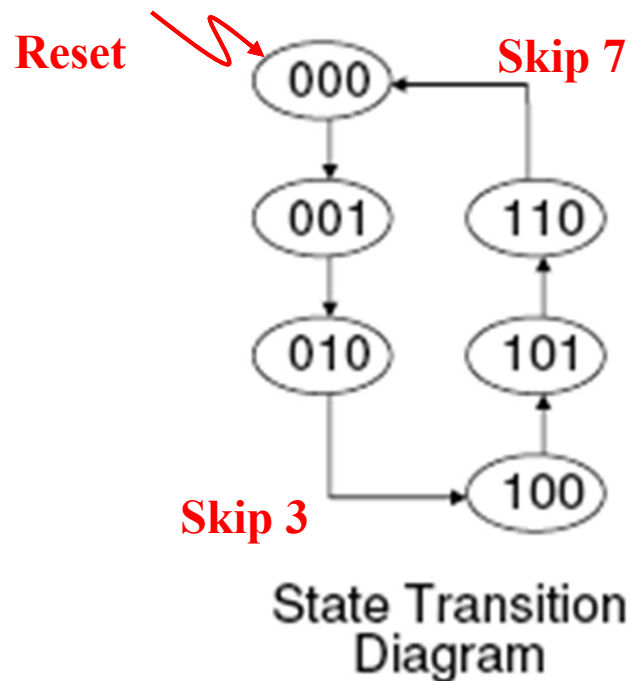


1. Obtain specification of the desired counter

Design a 3-bit binary UP synchronous counter that skips numbers 3 and 7. Use JK Flip-flops only

2. Create a state diagram from specification.
3. Create a state table from state diagram.

3 and 7 must be listed in the table even if the counter skips them (NS to be assigned Don't cares!)



Present State			Next State		
Q2	Q1	Q0	Q2+	Q1+	Q0+
0	0	0	0	0	1
0	0	1	0	1	0
0	1	0	1	0	0
0	1	1	X	X	X
1	0	0	1	0	1
1	0	1	1	1	0
1	1	0	0	0	0
1	1	1	X	X	X

State Transition Table

Special Counter using JK flip-flops

4. Choose flip-flop types (D, JK, or T)

Question asks for JK flip-flops

J	K	Q+
0	0	Q no change
0	1	0 reset
1	0	1 set
1	1	Q' toggle

Flip- Flop Excitation Tables

Q	Q+	J	K	
0	0	0	X	reset/hold
0	1	1	X	set/toggle
1	0	X	1	reset/toggle
1	1	X	0	set/hold

Q	J	K	Q+
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Design of next-state combinational logic

- Use excitation table of selected flip-flops to construct truth table

State Transition Table:

Present State			Next State			Flip-flop Input					
Q2	Q1	Q0	Q2+	Q1+	Q0+	J2	K2	J1	K1	J0	K0
0	0	0	0	0	1						
0	0	1	0	1	0						
0	1	0	1	0	0						
0	1	1	X	X	X						
1	0	0	1	0	1						
1	0	1	1	1	0						
1	1	0	0	0	0						
1	1	1	X	X	X						

Special Counter using JK flip-flops

5. Derive the next-state logic and output logic (if any) expressions.

J2 = ?

$Q_0 \backslash Q_2 Q_1$	00	01	11	10
0				
1				

J1 = ?

$Q_0 \backslash Q_2 Q_1$	00	01	11	10
0				
1				

J0 = ?

$Q_0 \backslash Q_2 Q_1$	00	01	11	10
0				
1				

K2 = ?

$Q_0 \backslash Q_2 Q_1$	00	01	11	10
0				
1				

K1 = ?

$Q_0 \backslash Q_2 Q_1$	00	01	11	10
0				
1				

K0 = ?

$Q_0 \backslash Q_2 Q_1$	00	01	11	10
0				
1				

Special Counter using JK flip-flops

6. Implement circuit described by expressions.

Special Counter using JK flip-flops



THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY

7. Check for the unused state.

State 011 → Next State ???

Last Step: Check for the unused states

State 111 → Next State???

Special Counter using JK flip-flops

4. Choose flip-flop types (D, JK, or T)

Question asks for JK flip-flops

J	K	Q+
0	0	Q no change
0	1	0 reset
1	0	1 set
1	1	Q' toggle

Flip- Flop Excitation Tables

Q	Q+	J	K	
0	0	0	X	reset/hold
0	1	1	X	set/toggle
1	0	X	1	reset/toggle
1	1	X	0	set/hold

Q	J	K	Q+
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Design of next-state combinational logic

- Use excitation table of selected flip-flops to construct truth table

State Transition Table:

Present State			Next State			Flip-flop Input			
Q2	Q1	Q0	Q2+	Q1+	Q0+	J2	K2	J1	K1
0	0	0	0	0	1	0	X	0	X
0	0	1	0	1	0	0	X	1	X
0	1	0	1	0	0	1	X	X	1
0	1	1	X	X	X	X	X	X	X
1	0	0	1	0	1	X	0	0	X
1	0	1	1	1	0	X	0	1	X
1	1	0	0	0	0	X	1	X	1
1	1	1	X	X	X	X	X	X	X

Q	Q+	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

Special Counter using JK flip-flops



5. Derive the next-state logic and output logic (if any) expressions

State Transition Table:

Present State			Next State			Flip-flop Input					
Q2	Q1	Q0	Q2+	Q1+	Q0+	J2	K2	J1	K1	J0	K0
0	0	0	0	0	1	0	X	0	X	1	X
0	0	1	0	1	0	0	X	1	X	X	1
0	1	0	1	0	0	1	X	X	1	0	X
0	1	1	X	X	X	X	X	X	X	X	X
1	0	0	1	0	1	X	0	0	X	1	X
1	0	1	1	1	0	X	0	1	X	X	1
1	1	0	0	0	0	X	1	X	1	0	X
1	1	1	X	X	X	X	X	X	X	X	X

J2

Q2\Q1	00	01	11	10
0	0	1	X	X
1	0	X	X	X

$$J2 = Q1$$

J1

Q2\Q1	00	01	11	10
0	0	X	X	0
1	1	X	X	1

$$J1 = Q0$$

J0

Q2\Q1	00	01	11	10
0	1	0	0	1
1	X	X	X	X

$$J0 = Q1'$$

K2

Q2\Q1	00	01	11	10
0	X	X	1	0
1	X	X	X	0

$$K2 = Q1$$

K1

Q2\Q1	00	01	11	10
0	X	1	1	X
1	X	X	X	X

$$K1 = 1$$

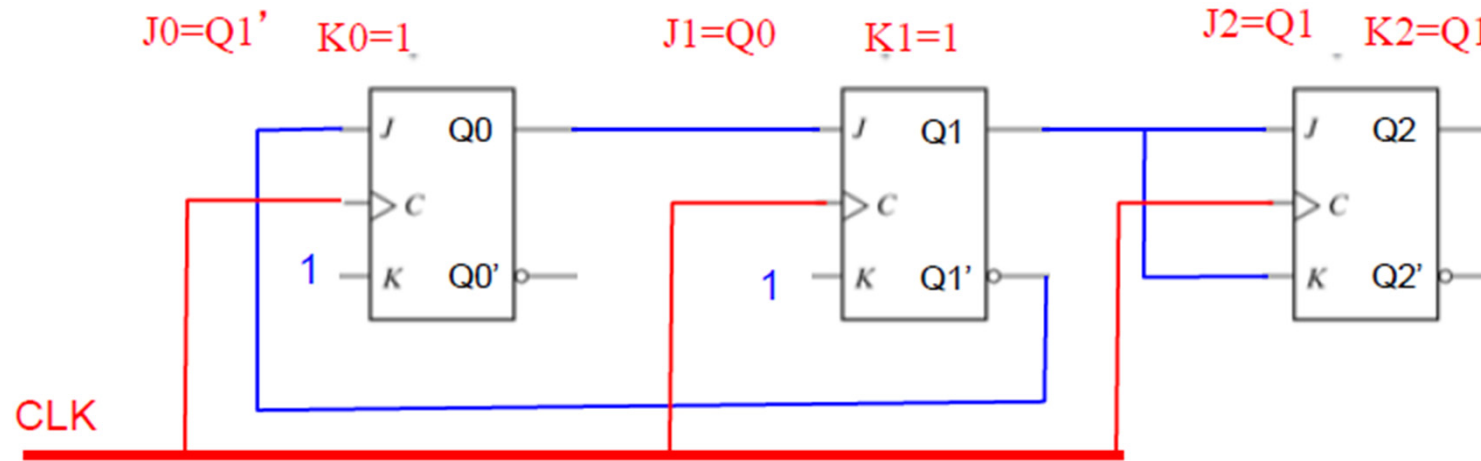
K0

Q2\Q1	00	01	11	10
0	X	X	X	X
1	1	X	X	1

$$K0 = 1$$

Special Counter using JK flip-flops

6. Implement circuit described by expressions



- Using more complicated flip-flops sometimes simplifies next-state logic. In times when all logic was implemented using TTL devices and T and JK flipflops were available as 'single' devices, they were used to achieve reduced component count.
- But nowadays D-type flip-flop is usually the basic component (VLSI chip design, programmable logic design, etc.)

Special Counter using JK flip-flops

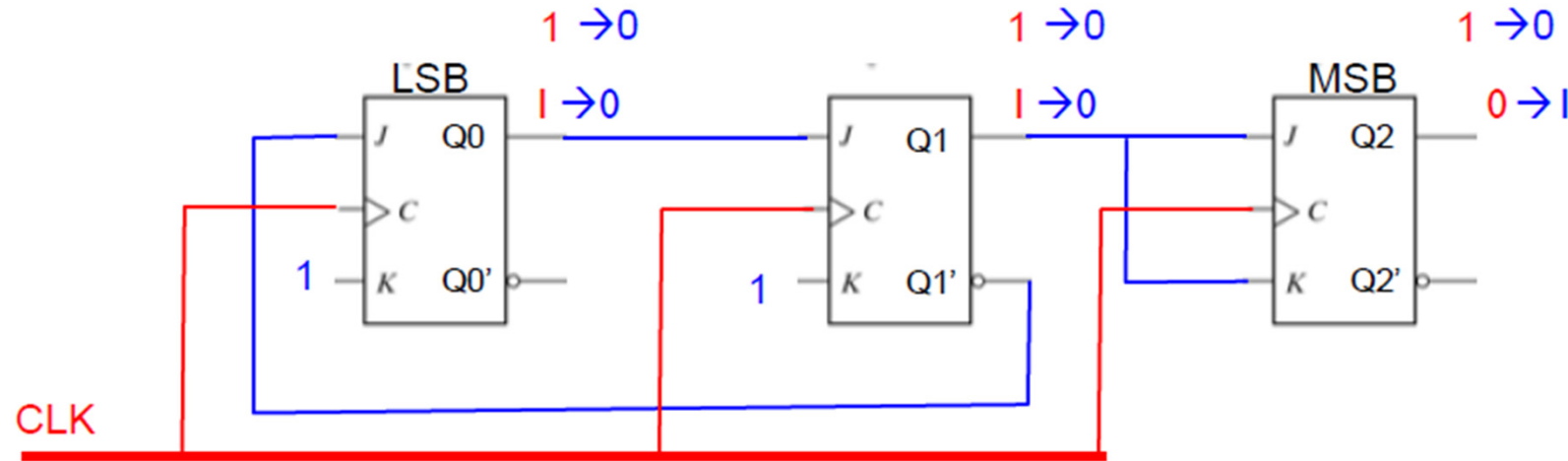


7. Check for the unused state

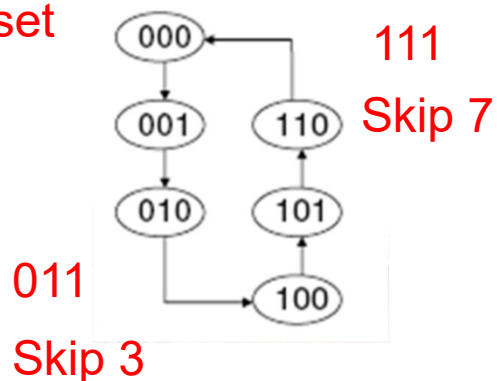
Last Step: Check for the unused states

State 011 → Next State ???

State 111 → Next State ???



Reset



If the counter finds itself accidentally in the unused state 011, it will then go into the state 100 at the next clock cycle.
That is, it will self correct!

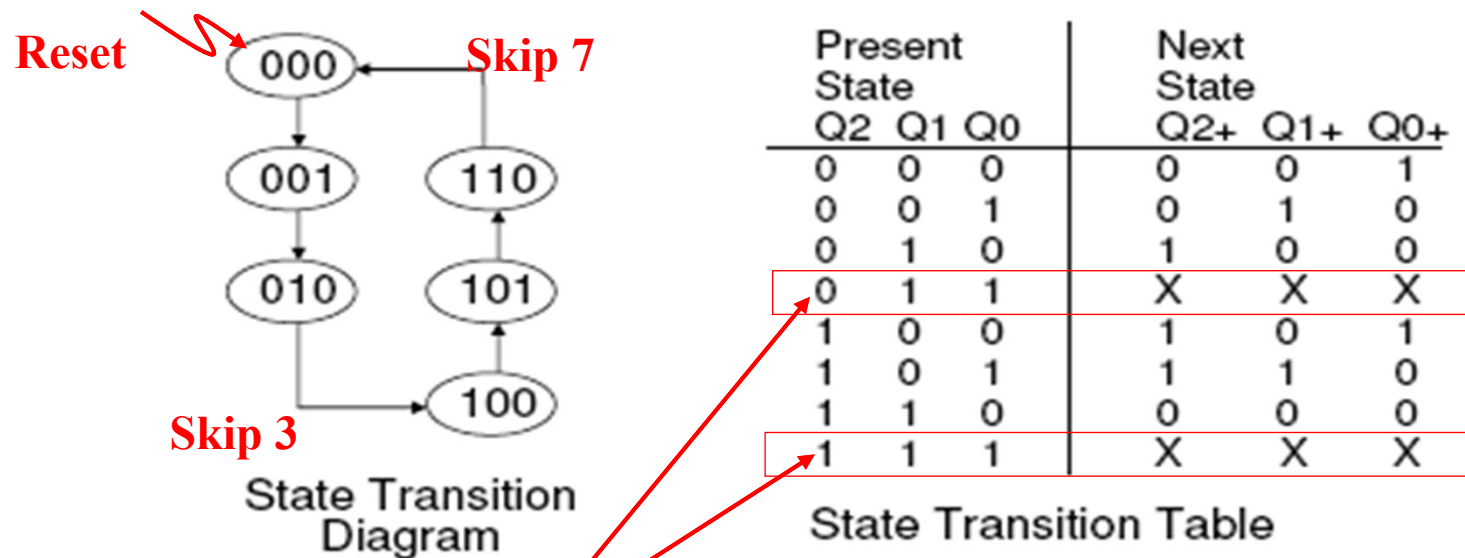
Same for the unused state 111; the counter will correct itself at the next clock cycle by moving into the valid state 000

- **In some cases, it may take more than one-clock cycle for the sequential circuit corrects itself**
- **We say that: the FSM (sequential circuit or in this case counter) is self-correcting after N clock cycles**
- **N can be 1 or more than 1 but N has to be a finite integer. Otherwise, the circuit is NOT a self-correcting circuit and has to be redesigned by forcing some or all of don't cares (X's) to 0 or 1**

Special Counter using T flip-flops



Build the same counter using T flip-flops



3 and 7 must be listed in the table even if the counter skips them (NS to be assigned Don't cares!)

Special Counter using T flip-flops

5. Derive the next-state logic and output logic (if any) expressions.

State Transition Table:

Present State			Next State			Flip-flop Input		
Q2	Q1	Q0	Q2+	Q1+	Q0+	T2	T1	T0
0	0	0	0	0	1			
0	0	1	0	1	0			
0	1	0	1	0	0			
0	1	1	X	X	X			
1	0	0	1	0	1			
1	0	1	1	1	0			
1	1	0	0	0	0			
1	1	1	X	X	X			

T	Q+
0	Q no change
1	Q' toggle

T flip-flop Excitation table

Q	Q+	T
0	0	0 hold
0	1	1 toggle
1	0	1 toggle
1	1	0 hold

T2 = ?

Q ₀	Q ₂ Q ₁			
	00	01	11	10
0				
1				

T1 = ?

Q ₀	Q ₂ Q ₁			
	00	01	11	10
0				
1				

T0 = ?

Q ₀	Q ₂ Q ₁			
	00	01	11	10
0				
1				

Special Counter using T flip-flops

6. Implement circuit described by expressions.

Special Counter using T flip-flops

7. Check for the unused state.

Last Step: Check for the unused states

State 011 → Next State ???

State 111 → Next State???

Special Counter using T flip-flops

5. Derive the next-state logic and output logic (if any) expressions

State Transition Table:

Present State			Next State			Flip-flop		
Q2	Q1	Q0	Q2+	Q1+	Q0+	T2	T1	T0
0	0	0	0	0	1	0	0	1
0	0	1	0	1	0	0	1	1
0	1	0	1	0	0	1	1	0
0	1	1	X	X	X	X	X	X
1	0	0	1	0	1	0	0	1
1	0	1	1	1	0	0	1	1
1	1	0	0	0	0	1	1	0
1	1	1	X	X	X	X	X	X

Q2Q1					
Q0		00	01	11	10
	0	0	1	1	0
	1	0	X	X	0

$$T2 = Q1$$

Q2Q1					
Q0		00	01	11	10
	0	0	1	1	0
	1	1	X	X	1

$$T1 = Q1 + Q0$$

Q2Q1					
Q0		00	01	11	10
	0	1	0	0	1
	1	1	X	X	1

$$T0 = Q1'$$

T	Q+
0	Q no change
1	Q' toggle

T flip-flop Excitation table

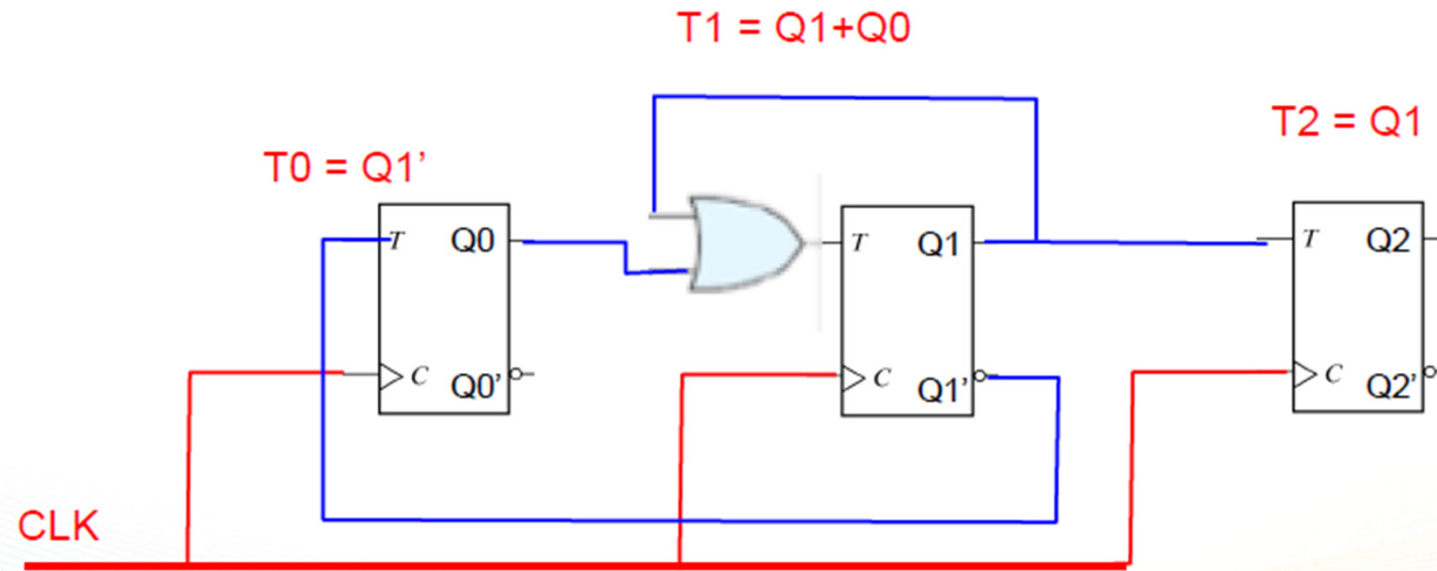
Q	Q+	T
0	0	0 hold
0	1	1 toggle
1	0	1 toggle
1	1	0 hold

Special Counter using T flip-flops



THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY

6. Implement circuit described by expressions



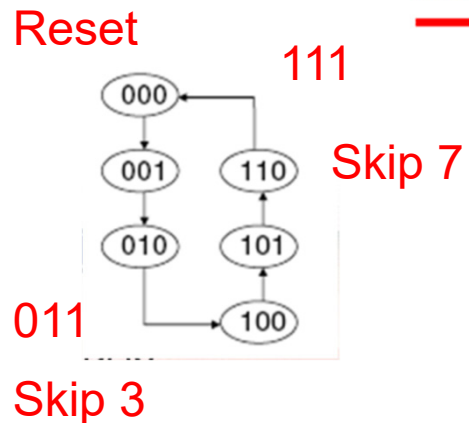
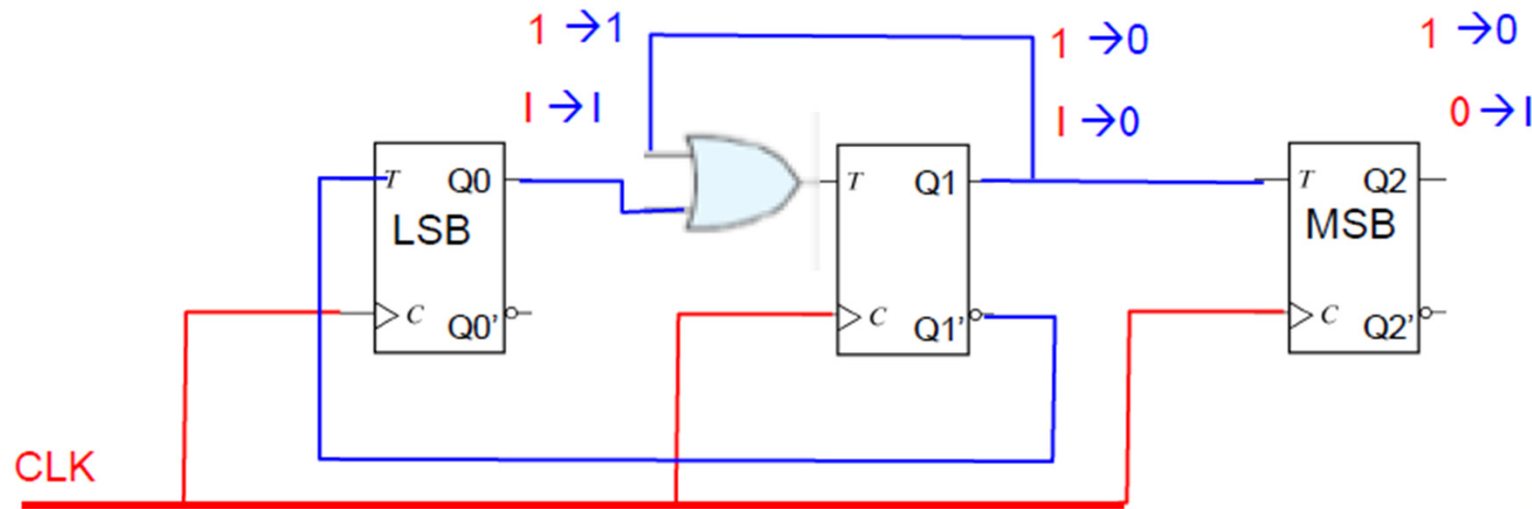
Special Counter using T flip-flops

7. Check for the unused state

State 011 → Next State ???

Last Step: Check for the unused states

State 111 → Next State???



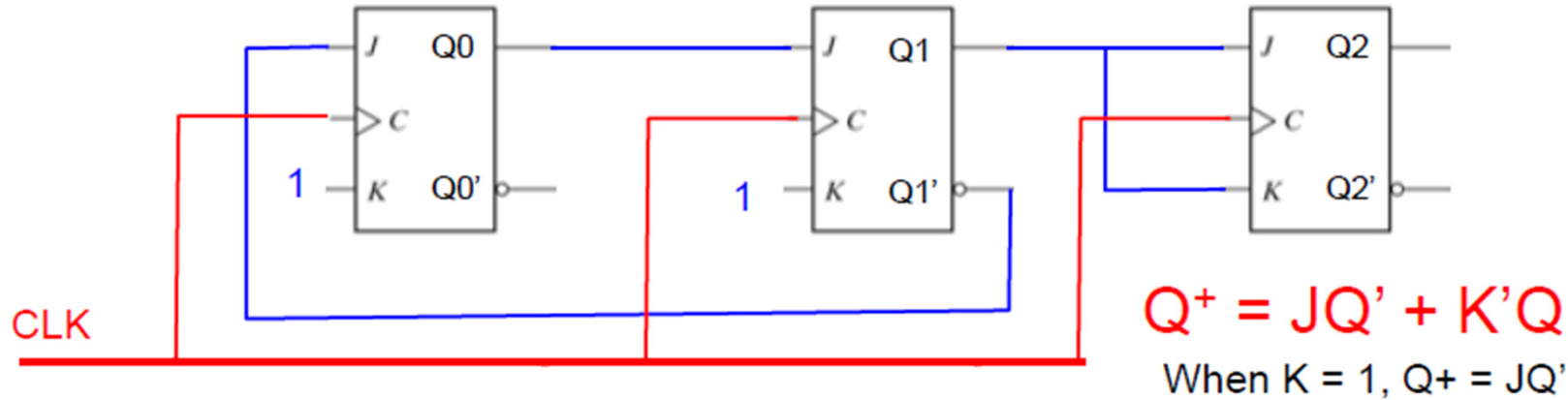
If the counter finds itself accidentally in the unused state 011, it will then go into the state 101 at the next clock cycle
That is, it will self correct!

Same for the unused state 111; the counter will correct itself at the next clock cycle by moving into the valid state 001

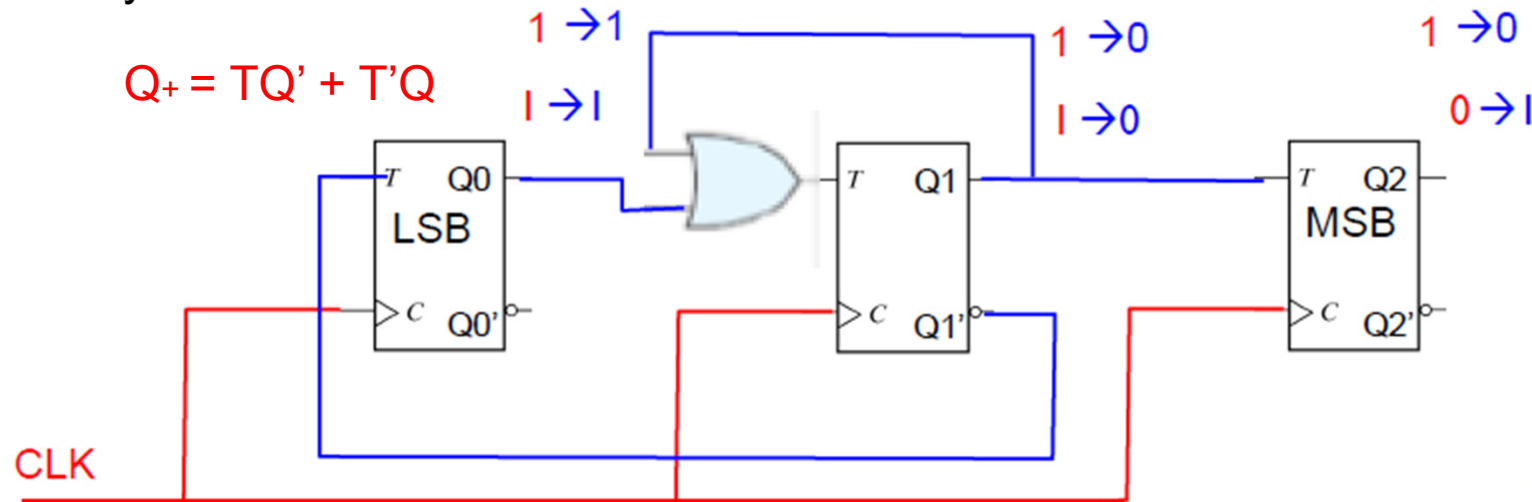
Special Counter using T flip-flops



Compare two counters



Are both the same? Why?

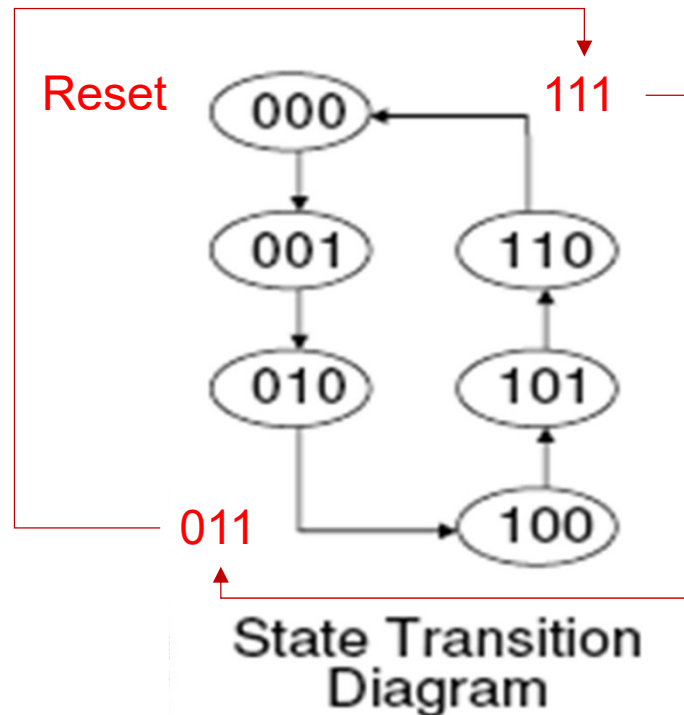


Worst Case Scenario

Why do we need to check for the unused states???

Because the FSM may cycle from one unused state to another and if it does, it will never recover (correct) itself unless you reset it!

If this condition is likely to occur, you will have to force some or all of the “don’t cares” to either 0 or 1 (valid state)



Present State			Next State		
Q2	Q1	Q0	Q2+	Q1+	Q0+
0	0	0	0	0	1
0	0	1	0	1	0
0	1	0	1	0	0
0	1	1	X	X	X
1	0	0	1	0	1
1	0	1	1	1	0
1	1	0	0	0	0
1	1	1	X	X	X

State Transition Table

1. Obtain specifications of the desired counter
2. Create a state diagram from specification.
3. Create a state table from state diagram.
4. Choose flip-flop types (D, JK, or T) if the question does not specify any flip-flop type
5. Derive the next-state logic and output logic (if any) expressions.
6. Implement circuit described by expressions.
7. Check for unused states (if any) to see whether the counter is self-correcting. If it never self-corrects then redesign the counter by forcing select don't cares to 0 or 1



香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

Thank you for listening !

